

A Computer Networking Individual Project

Multi-Threaded Web Server

Project Report

YANG Xikun

Apr 2025, revised Sept 2025

Important Information

Please make sure to read this page first.

- The server part of this project is implemented entirely in **Java**.
- I designed a **GUI window interface**. The window uses the **Javax Swing** framework and can run on all JVMs.
- Some components of the JVM have issues. They might be legacy issues or implementation flaws. **This doesn't mean there is a problem with my implementation.**
- When you run the Server, you will first see the **Main Window**. The Main Window includes the **Server Console**, where all Logs are showed.
- A small part on the right side of the Main Window is a **Clients Simulator**. You can quickly communicate with the Server as a Client without opening a browser here.
- The Server and Clients Simulator are independent of each other.
- According to the instruction document, I also designed a **Client Webpage** where you act entirely as a Client there.
- Please open the Server, and then you can also request files from the Server on this webpage. This seems more reasonable.
- I submitted the entire project folder, and this report is also in the project folder. If you need to, simply drag this folder into IntelliJ IDEA and you can edit it.
- All the server's source code is in the **src/** directory. The entry of the program is **src/com/frankyang/polyu/comp2322/webserver/WebServerApp.java**.
- The Client Webpage is in the **client-website/** folder.
- I also compiled the server into a **.jar** file. That is, **WebServer.jar**.

Run This Program

You have several ways to run this program. In any case, please install **Java 16** first.

- You can download Java 16 from [here](#).
- You can download the latest version of Java from [here](#).
- If you are using **PolyU Lab computers** for evaluation, Java 17 or Java 18 should be installed by default, and you do not need to install Java 16 again.

You can run the **Web Server** in the following ways. No matter which way you choose, please note that you need to open the entire folder I submitted.

- Open the entire folder using IntelliJ IDEA. Then run the following class:
`src/com/frankyang/polyu/comp2322/webserver/WebServerApp.java`
- **Run the .jar file.** Use `cd` in the command line tool to switch the working directory to the folder I submitted. Then use `java -jar WebServerApp.jar` to start the Web Server.
- You can also simply drag the `WebServerApp.java` file into the command line tool to run it. In that case, the command should be `java -jar <path to WebServerApp.jar>`.

You can run the Client in the following way:

- Use the Client Simulator in the Web Server's Main Window.
- Open `client-website/client-index.html`. You can also start the Client Website by clicking the **Test via Browser** button in the Main Window.

You need to open the Server first; or, no Client can communicate with the Server.

The GitHub repo of this project is:

<https://github.com/FrankYang0610/Multi-Threaded-Web-Server>

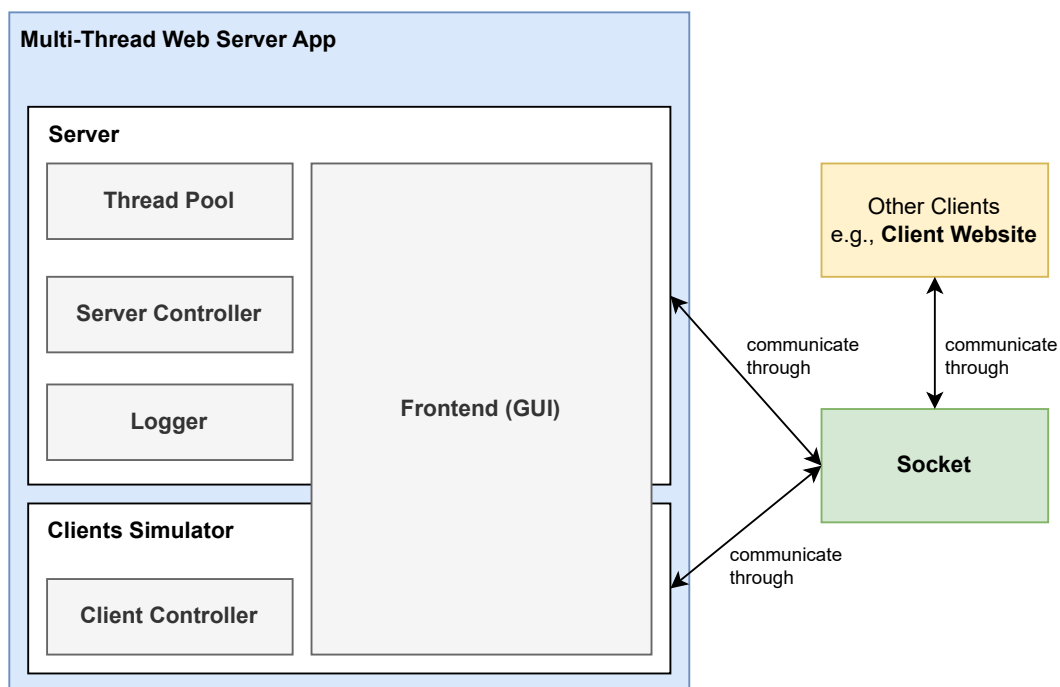
Program Structure

Please ensure you have read the **Important Information**. You should already know that the program integrates the **Server** and the **Clients Simulator**.

Here, I would like to emphasize once again that although the Server and Clients Simulator are together on the GUI, the communication between the Clients Simulator and the Server is still completed via the socket. **The two are completely independent both logically and in fact. The reason for this design is to facilitate better testing.**

This program uses the **MVC (Model-View-Controller)** design concept. The frontend parts and backend parts are completely separated, and there are no problematic designs like circular dependencies.

Figuratively speaking, the structure of this program is:



(Structure of the program)

Key Assumptions

Here are the assumptions used by the program:

- All constants are stored in `src/.../config/Config.java`.
- The default communication uses the `HTTP/1.1` protocol.
- It is recommended to use the Server and client on the **same device**. Of course, you can also use the Server and Client **on several devices** if you know the Server IP address and port.
- The default Server IP address is `127.0.0.1` and the port is `8080`. If the port is occupied, the program will try to find a new available port.
- If you need cross-device communication, you need to **recompile** the program.
- You can use the resources in the `resources/` folder to test the server. You are also encouraged to add files to the `resources/` folder yourself to test the server's generality.
- For safety reasons, accessing files **outside** the `resources/` folder will result in a 403 Forbidden error.
- Please try to ensure that file names **do not** contain spaces and only include `A-Z`, `a-z`, and `0-9`. It is not impossible to handle file names with special characters, but there may be potential risks.
- Access to `resources/forbidden.txt` is **forbidden** by default. This is already shown in `Config.java`.
- The server supports handling **text and images** requests. Supported file types include: `.txt`, `.md`, `.c`, `.cpp`, `.h`, `.hpp`, `.java`, `.py`, `.js`, `.ts`, `.html`, `.htm`, `.css`, `.swift`, `.kt`, `.jpg`, `.jpeg`, and `.png`
- `.pdf` file is **not** supported.

Test Resources

As mentioned in Key Assumptions, the root directory of the Server's resources is in the **resources/** folder. In this folder, I have added some files to facilitate your testing.

These files include:

- `text.txt` – A short text introducing red-crowned cranes, used to test text requests.
- `longtext.txt` – A long article from the Wikipedia on red-crowned cranes, used to test long text requests.
- `forbidden.txt` – A file with access denied, used to trigger the 403 Forbidden error.
- `flying.jpg` – A photo of two red-crowned cranes in flight, used to test photo requests.
- `flying.pdf` – The PDF version of `flying.jpg`, used to test the 415 Unsupported Media Type error.
- `macaw.jpeg` – A photo of a blue-and-gold macaw, used to test requests for `.jpeg` images.

Note that when entering the path, please ensure it starts with a `/`, for example, `/text.txt` or `/flying.jpg`.

Example Logs

According to the requirements of the instruction document, two complete log files are displayed. **They are all stored in the log/ folder.** You can fully reproduce these two logs based on Complete Journey I – Testing All Functionalities and Complete Journey II – Testing Most Functionalities.

Browser Restrictions

It must be pointed out that due to **browser restrictions**, some things cannot be implemented:

- The browser will automatically check the syntax of HTTP requests. An incorrect HTTP request will not be sent to the Server at all. Therefore, it's hard to trigger a **400 Bad Request** error via the browser.
- The browser will automatically correct the path. Paths containing “.” and other special meanings will be corrected. Finally, the path in the request sent to the Server is always compliant. Therefore, it's difficult to trigger a **403 Forbidden** error via directory traversal attacks.

You can still verify that the server is perfect!

- For the difficulties in the above two browser tests, you can easily solve them using the Main Window's Clients Simulator. Please refer to the following sections.

Scoring criteria

The Web Server program meets **all** the requirements. You can quickly find the parts of the program that meet each scoring criterion here.

Scoring criterion	Corresponding source code file (in <code>src/com/frankyang/polyu/comp2322/webserver/</code>)	Page number in the later part of the report
Proper request and response message exchanges (5 marks)	(Server Frontend) <code>gui/MainWindow.java</code> <code>gui/ChooseFileDialog.java</code> (Server Backend) <code>servercontroller/ServerController.java</code> <code>threadpool/ServerThreadPool.java</code> <code>http/HttpRequestHandler.java</code> <code>http/HttpRequestProcessor.java</code> <code>http/HttpResponse.java</code> (Clients Simulator) <code>gui/ResponseDialogManager.java</code> <code>clientcontroller/ClientController.java</code>	P11 ~ P15
GET command for both text files and image files (10 marks)	(Server Frontend) <code>gui/MainWindow.java</code> <code>gui/ChooseFileDialog.java</code> (Server Backend) <code>http/HttpRequestProcessor.java</code> (Clients Simulator) <code>gui/ChooseFileDialog.java</code> <code>clientcontroller/ClientController.java</code>	P16 P18 <u>Handle GET and HEAD requests</u>
HEAD command (5 marks)	(Server Frontend) <code>gui/MainWindow.java</code> <code>gui/ChooseFileDialog.java</code> (Server Backend) <code>http/HttpRequestHandler.java</code>	P17 P18 <u>Handle GET and HEAD requests</u>

Scoring criteria

Six types of response statuses (30 marks)	(Server Backend) http/HttpRequestProcessor.java http/HttpResponse.java (Clients Simulator) gui/ChooseFileDialog.java clientcontroller/ClientController.java	P19 ~ P24
Handle Last-Modified and If-Modified-Since header fields (10 marks)	(Server Backend) http/HttpRequestProcessor.java (Clients Simulator) gui/ChooseFileDialog.java clientcontroller/ClientController.java	P25 ~ P27
Handle Connection header field (10 marks)	(Server Backend) threadpool/ServerThreadPool.java http/HttpRequestHandler.java http/HttpRequestProcessor.java (Clients Simulator) gui/ChooseFileDialog.java clientcontroller/ClientController.java	P28 ~ P29
A good summary of your design and implementation (10 marks)	/	P2 <u>Important Information</u> P4 <u>Program Structure</u> P5 <u>Key Assumptions</u> P30 ~ 33 <u>Summary of Implementation Details</u>

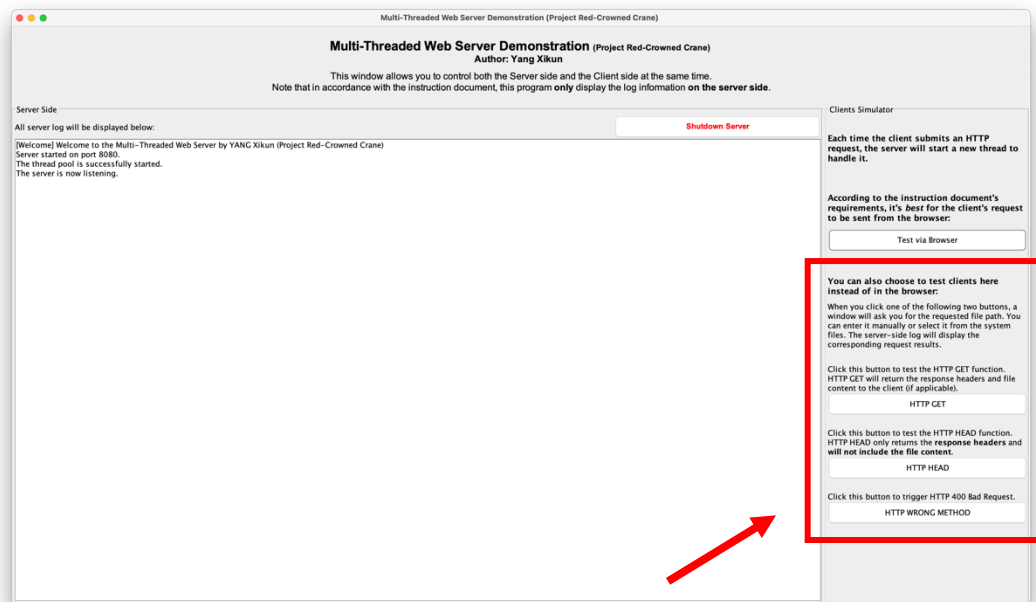
Scoring criteria

A full demonstration of executing your program and screen capturing of results of all functions (5 marks)	/	P34~ P45 <u>Complete Journey I – Testing All Functionalities</u> P46 ~ P52 <u>Complete Journey II – Testing Most Functionalities</u>
A complete log file (5 marks)	(Stored in the project folder) logs/log-complete-journey-I.txt logs/log-complete-journey-I.txt	P34 ~ P45 <u>Complete Journey I – Testing All Functionalities</u> P46 ~ P52 <u>Complete Journey II – Testing Most Functionalities</u>
A clear README text file (5 marks)	(Stored in the project folder) README.md	/
Quality of your project's source code (5 marks)	/	/

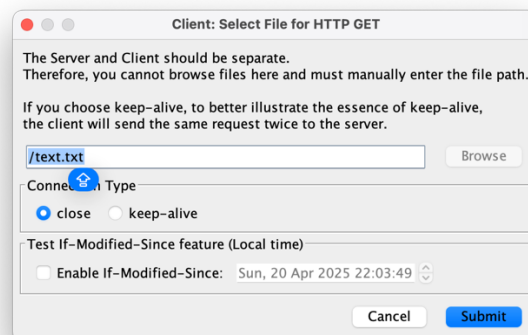
Proper request and response message exchanges

The request is sent by the Client.

- In the Main Window, You can click the **HTTP GET** or **HTTP HEAD** button in the Clients Simulator to indicate that you are a client wishing to send a GET or HTTP request to the server.



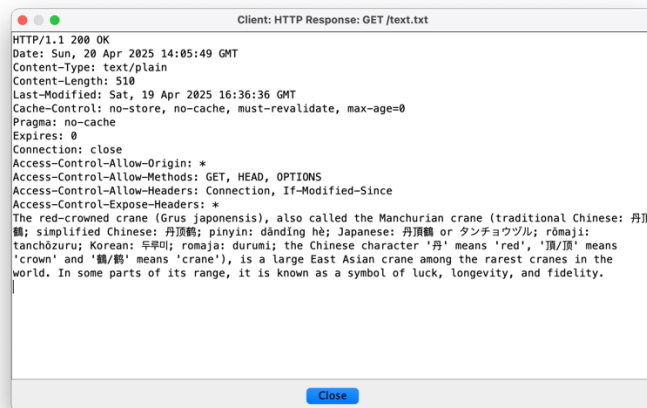
- A **Choose File Dialog** window will then appear. Here, you can select which file to request from the Server, the type of connection to use (close or keep-alive), and whether to use If-Modified-Since.



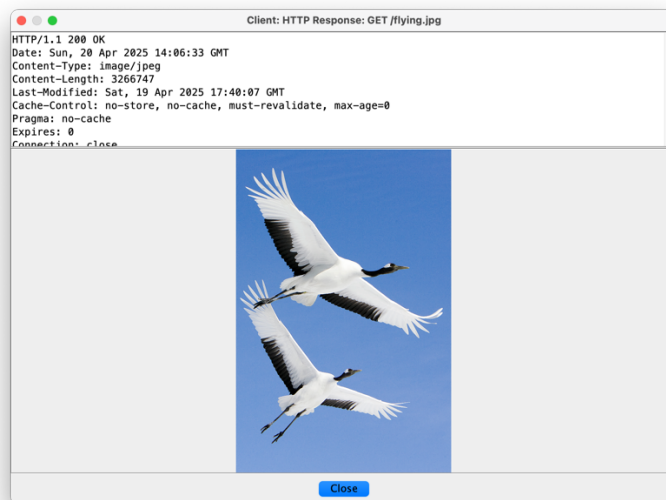
- Click **Submit** to send the request.

Proper request and response message exchanges

- The Server will then return the **response**, including the **headers** and the **response body**.



- Of course, if it's an image, the program will render the image.



- The Server will also record the status of this request:

```
[Welcome] Welcome to the Multi-Threaded Web Server by YANG Xikun (Project Red-Crowned Crane)
Server started on port 8080.
The thread pool is successfully started.
The server is now listening.
127.0.0.1:56273 [Sun, 20 Apr 2025 14:05:49 GMT] "GET /text.txt HTTP/1.1" 200 OK (Server: 127.0.0.1:8080) 510 bytes
127.0.0.1:56403 [Sun, 20 Apr 2025 14:06:33 GMT] "GET /flying.jpg HTTP/1.1" 200 OK (Server: 127.0.0.1:8080) 3266747 bytes
```


Proper request and response message exchanges

- The implementation is like this:
 - The Server listens on a fixed port (8080 by default).
 - The Client frontend class `ChooseFileDialog` collects the Client's request information and then passes it to the backend `ClientController` class.
 - The `ClientController` creates a socket to send a request to the Server.
 - The Server uses `HttpRequestHandler` to arrange a new thread for the request, then uses `HttpRequestProcessor` to handle the request and generate a response.
 - The `ClientController` receive the response and shows its content via another frontend class `ResponseDialogManager`.
- You can check the `HttpRequestProcessor` class carefully. The `handleRequest()` method is responsible for the main tasks, processing the request and sending the response.
- The methods `sendOK()`, `sendError()`, etc., are used to actually send the response.

```
private void sendOK(OutputStream out, String contentType, File file) throws IOException {
    String header = "HTTP/" + httpVersion + " 200 OK\r\n" +
        "Date: " + DateTimeManager.formatHttpDate(timestamp: System.currentTimeMillis()) + "\r\n" +
        "Content-Type: " + contentType + "\r\n" +
        "Content-Length: " + file.length() + "\r\n" +
        "Last-Modified: " + DateTimeManager.formatHttpDate(timestamp: file.lastModified()) + "\r\n" +
        "Cache-Control: no-store, no-cache, must-revalidate, max-age=0\r\n" +
        "Pragma: no-cache\r\n" +
        "Expires: 0\r\n" +
        "Connection: " + (keepAlive ? "keep-alive" : "close") + "\r\n" +
        "Access-Control-Allow-Origin: *\r\n" +
        "Access-Control-Allow-Methods: GET, HEAD, OPTIONS\r\n" +
        "Access-Control-Allow-Headers: Connection, If-Modified-Since\r\n" +
        "Access-Control-Expose-Headers: *\r\n" +
        "\r\n";
    out.write(b: header.getBytes());

    try (BufferedInputStream bis = new BufferedInputStream(in: new FileInputStream(file))) {
        byte[] buffer = new byte[Config.BUFFER_SIZE];
        int bytesRead;
        while ((bytesRead = bis.read(b: buffer)) != -1) {
            out.write(b: buffer, off: 0, len: bytesRead);
        }
    }

    out.flush();
}
```

Proper request and response message exchanges

- The implementation of `ClientServer` is relatively complex, so here is a brief introduction.
 - The public methods `sendGetRequest()` and `sendHeadRequest()` are given to the frontend.
 - The `sendRequest()` method is used to send content to the socket (i.e., to the Server).

```
private void sendRequest(String method, String path, boolean keepAlive, Date ifModifiedSince, Socket socket)
    throws IOException {
    String encodedPath = encodePath(path);
    String connectionType = keepAlive ? "keep-alive" : "close";

    OutputStream out = socket.getOutputStream();

    StringBuilder requestBuilder = new StringBuilder();
    requestBuilder.append(method).append(" ").append(encodedPath).append(" HTTP/1.1\r\n");
    requestBuilder.append("Host: ").append(serverIP).append(":").append(serverPort).append("\r\n");
    requestBuilder.append("Connection: ").append(connectionType).append("\r\n");
    requestBuilder.append("User-Agent: client\r\n");

    if (ifModifiedSince != null) {
        String formattedDate = DateTimeManager.formatHttpDate(timestamp: ifModifiedSince.getTime());
        requestBuilder.append("If-Modified-Since: ").append(formattedDate).append("\r\n");
    }

    requestBuilder.append("\r\n");
    String request = requestBuilder.toString();

    out.write(request.getBytes());
    out.flush();
}
```

- When the response from the Server is received, the methods `processResponse()`, `processTextResponse()`, and `processImageResponse()` will process and display the response. Please read the code.

Proper request and response message exchanges

- When you are using `client-website/client-index.html`, things are slightly different.
 - Please check `clients-website/client-script.js`, which includes the logic for sending requests to the Server and receiving responses from the Server.

GET command for both text files and image files

- Please read the `ClientController` class in detail, especially the `sendGetRequest()` method.

```
public void sendGetRequest(String path, boolean keepAlive, Date ifModifiedSince) throws IOException {
    if (keepAlive) {
        // To test keep-alive connection, the client will send the same request to the server twice.
        try (Socket socket = new Socket( host: serverIP, serverPort)) {
            sendRequest( method: "GET", path, keepAlive, ifModifiedSince, socket);
            processResponse(socket, method: "GET", path);

            try {
                Thread.sleep( millis: 1000); // 1s
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            sendRequest( method: "GET", path, keepAlive, ifModifiedSince, socket);
            processResponse(socket, method: "GET", path);
        }
    } else {
        try (Socket socket = new Socket( host: serverIP, serverPort)) {
            sendRequest( method: "GET", path, keepAlive, ifModifiedSince, socket);
            processResponse(socket, method: "GET", path);
        }
    }
}
```

- The Server's backend will send these contents. Please read the `HttpRequestProcessor` class. Please also refer to [Handle GET and HEAD requests](#).
- On the `client-index` website, you can click the **Send GET Request** button. The `client-script.js` shows the logic for sending requests.

```
fetch(inputs.url, init: {
    method,
    headers,
    cache: 'no-store'
}) Promise<Response>
    .then(async response : Response => {
```

HEAD command

- Please read the `ClientController` class in detail, especially the `sendHeadRequest()` method.

```
public void sendHeadRequest(String path, boolean keepAlive, Date ifModifiedSince) throws IOException {
    if (keepAlive) {
        // To test keep-alive connection, the client will send the same request to the server twice.
        try (Socket socket = new Socket( host: serverIP, serverPort)) {
            sendRequest( method: "HEAD", path, keepAlive, ifModifiedSince, socket);
            processResponse(socket, method: "HEAD", path);

            try {
                Thread.sleep( millis: 1000); // 1s
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            sendRequest( method: "HEAD", path, keepAlive, ifModifiedSince, socket);
            processResponse(socket, method: "HEAD", path);
        }
    } else {
        try (Socket socket = new Socket( host: serverIP, serverPort)) {
            sendRequest( method: "HEAD", path, keepAlive, ifModifiedSince, socket);
            processResponse(socket, method: "HEAD", path);
        }
    }
}
```

- The Server's backend will send the HEAD response. Please read the `HttpRequestProcessor` class. Please also refer to [Handle GET and HEAD requests.](#)

```
private void sendHead(OutputStream out, String contentType, long contentLength, String lastModifiedStr)
    throws IOException {
    String header = "HTTP/" + httpVersion + " 200 OK\r\n" +
        "Date: " + DateTimeManager.formatHttpDate( timestamp: System.currentTimeMillis()) + "\r\n" +
        "Content-Type: " + contentType + "\r\n" +
        "Content-Length: " + contentLength + "\r\n" +
        "Cache-Control: no-store, no-cache, must-revalidate, max-age=0\r\n" +
        "Pragma: no-cache\r\n" +
        "Expires: 0\r\n" +
        "Connection: " + (keepAlive ? "keep-alive" : "close") + "\r\n" +
        "Access-Control-Allow-Origin: *\r\n" +
        "Access-Control-Allow-Methods: GET, HEAD, OPTIONS\r\n" +
        "Access-Control-Allow-Headers: Connection, If-Modified-Since\r\n" +
        "Access-Control-Expose-Headers: *\r\n" +
        "\r\n";
    out.write( b: header.getBytes());
    out.flush();
}
```

- On the `client-index` website, you can click the **Send HEAD Request** button. The `client-script.js` shows the logic for sending requests.

```
fetch(inputs.url, init: {
    method,
    headers,
    cache: 'no-store'
}) Promise<Response>
    .then(async response : Response => {
```

Handle GET and HEAD requests

The `HTTPRequestProcess` class demonstrates how the Server backend responds to GET and HEAD requests and returns the correct response. A brief introduction to the processing flow is provided below.

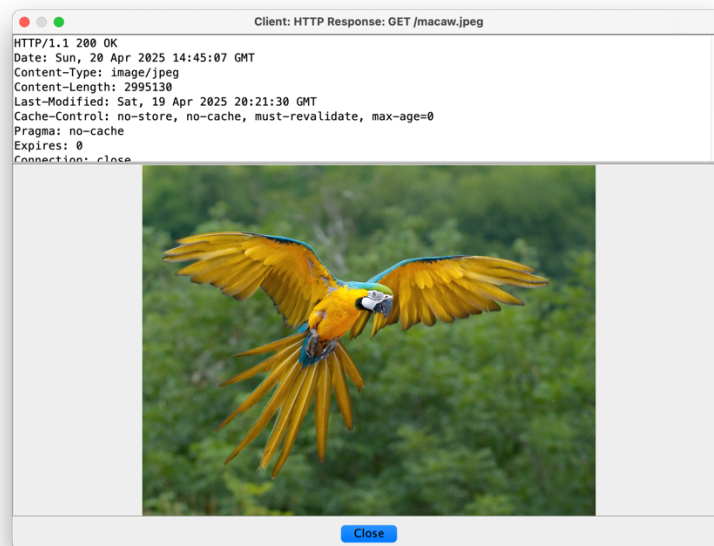
- When the `HTTPResponseHandler` of a **request processing thread** initiates an `HTTPResponseProcessor` object, the `HTTPResponseProcessor` first reads the content of the request from the input stream.
- The `HTTPRequestProcessor` uses a `HashMap` to read the header fields.
- Check the `Connection` header to determine whether the response should maintain the connection.
- Check if the method is GET, HEAD, or OPTIONS, otherwise, send a **400 Bad Request** error.
- Check the file path requested in the HTTP request. If there are any invalid parameters, send a **400 Bad Request** error.
- Check if the requested file is in a forbidden directory. If it is, send a **403 Forbidden** error.
- Convert the file path to a canonical file path. If directory traversal is detected, send a **403 Forbidden** error.
- If the file is not found, send a **404 File Not Found** error.
- If the file is found but the file type is not supported, send a **415 Unsupported Media Type** error.
- If the user has enabled the `If-Modified-Since` header, determine the relationship between the requested file's last modified date and the `If-Modified-Since` date. If the last modified date is earlier than the `If-Modified-Since` date, send a **304 Not Modified**.
- If the method is HEAD, send a **200 OK** without the requested file.
- If the method is GET, send a **200 OK** with the requested file.
- Both text and images are sent in byte form via a `BufferedInputStream`.

Six types of response statuses

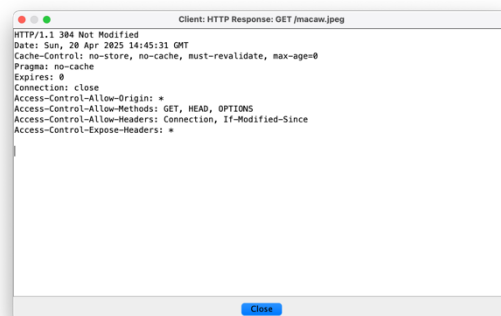
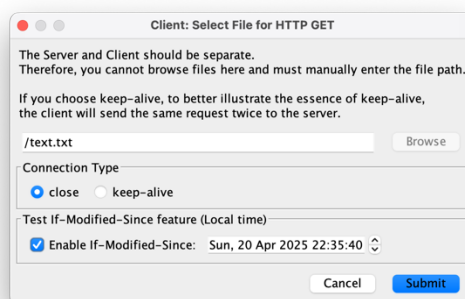
The methods to trigger each error is **highlighted**.

First, please be aware of the **trigger conditions** for the six response statuses.

- **200 OK:** Correct requests will return 200 OK. Please ensure your file path is correct, starts with a "/" and exists in the resources folder, and the file type is appropriate (see **Key Assumptions**).

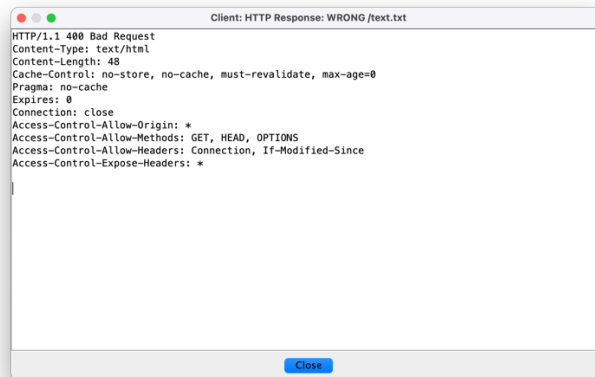


- **304 Not Modified:** You will only receive a 304 Not Modified if you have **selected Enable If-Modified-Since** while the file's last modified time is **earlier** than the time you selected in the If-Modified-Since datetime selection box.

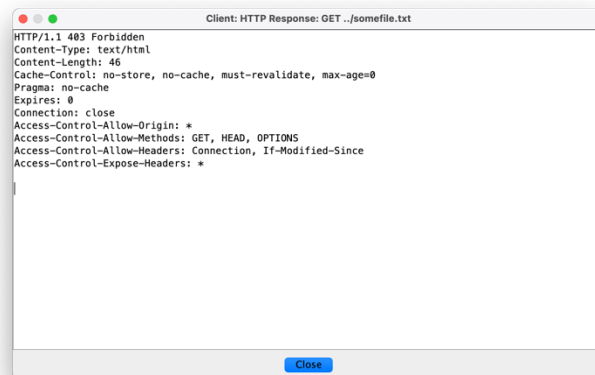


Six types of response statuses

- **400 Bad Request:** You cannot test this in the browser (see **Browser Restrictions**). However, you can click the **HTTP WRONG METHOD** button in the Clients Simulator and **send wrong HTTP requests** to trigger this error.

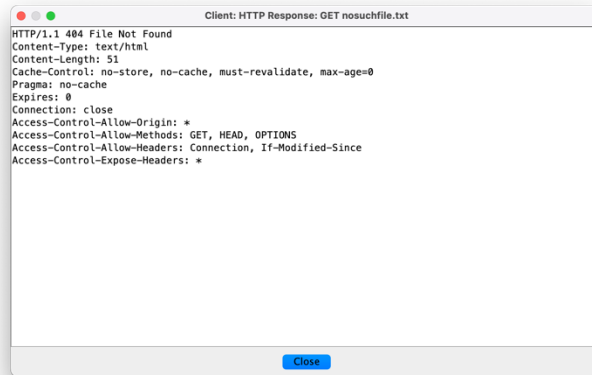


- **403 Forbidden:** Requesting for content outside the `resources/` folder (using `..` to bypass `resources/`) or accessing `resources/forbidden.txt` shall trigger this error.

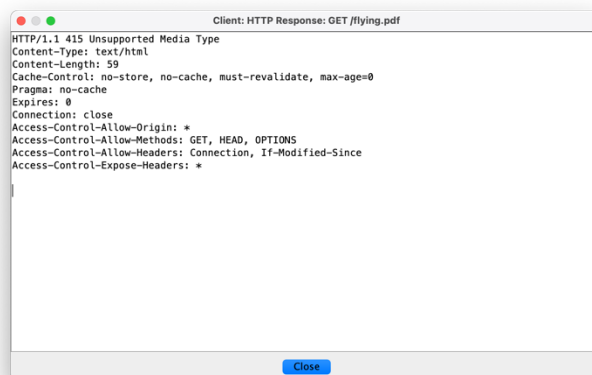


Six types of response statuses

- **404 File Not Found:** Requesting for a non-existent file will trigger this error. Assuming there is no `nosuchfile.txt` in the `resources/` folder, requesting for `nosuchfile.txt` will trigger this error.

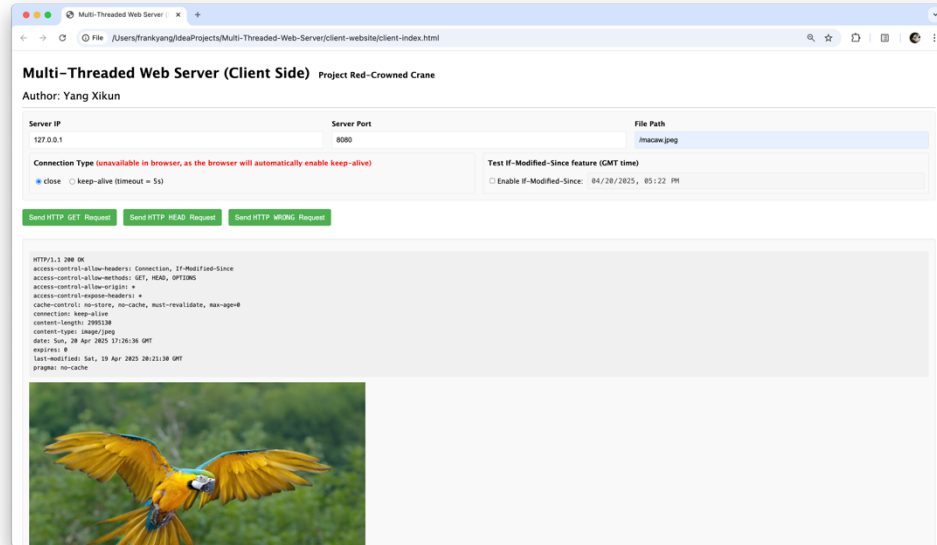


- **415 Unsupported Media Type:** Requesting for a file with an unsupported file type will trigger this error. It's recommended to request for `flying.pdf`, which is in `resources/` and can directly trigger this error.

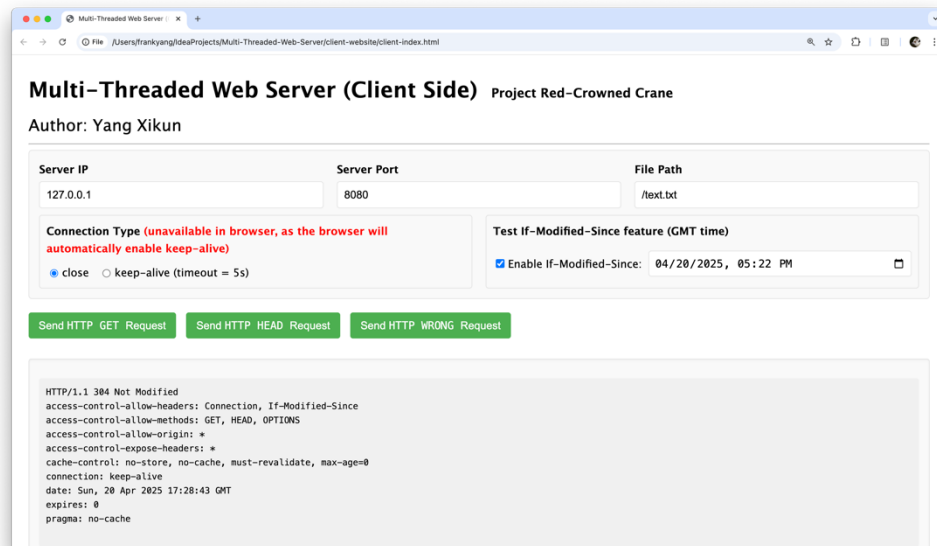


Six types of response statuses

- You can test errors except the 400 Bad Request on the Client Website.



(HTTP 200 OK)



(HTTP 304 Not Modified)

Six types of response statuses

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /forbidden.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is enabled with a date of 04/20/2025, 05:22 PM. The response status is 'HTTP/1.1 403 Forbidden'. The response headers are: access-control-allow-headers: Connection, If-Modified-Since; access-control-allow-methods: GET, HEAD, OPTIONS; access-control-allow-origin: *; access-control-expose-headers: *; cache-control: no-store, no-cache, must-revalidate, max-age=0; connection: keep-alive; content-length: 46; content-type: text/html; expires: 0; pragma: no-cache.

Multi-Threaded Web Server (Client Side) Project Red-Crowned Crane
Author: Yang Xikun

Server IP: 127.0.0.1 Server Port: 8080 File Path: /forbidden.txt

Connection Type (unavailable in browser, as the browser will automatically enable keep-alive)
☒ close ☐ keep-alive (timeout = 5s)

Test If-Modified-Since feature (GMT time)
☒ Enable If-Modified-Since: 04/20/2025, 05:22 PM

Send HTTP GET Request Send HTTP HEAD Request Send HTTP WRONG Request

```
HTTP/1.1 403 Forbidden
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 46
content-type: text/html
expires: 0
pragma: no-cache
```

(HTTP 403 Forbidden)

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /nosuchfile.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is enabled with a date of 04/20/2025, 05:22 PM. The response status is 'HTTP/1.1 404 File Not Found'. The response headers are: access-control-allow-headers: Connection, If-Modified-Since; access-control-allow-methods: GET, HEAD, OPTIONS; access-control-allow-origin: *; access-control-expose-headers: *; cache-control: no-store, no-cache, must-revalidate, max-age=0; connection: keep-alive; content-length: 51; content-type: text/html; expires: 0; pragma: no-cache.

Multi-Threaded Web Server (Client Side) Project Red-Crowned Crane
Author: Yang Xikun

Server IP: 127.0.0.1 Server Port: 8080 File Path: /nosuchfile.txt

Connection Type (unavailable in browser, as the browser will automatically enable keep-alive)
☒ close ☐ keep-alive (timeout = 5s)

Test If-Modified-Since feature (GMT time)
☒ Enable If-Modified-Since: 04/20/2025, 05:22 PM

Send HTTP GET Request Send HTTP HEAD Request Send HTTP WRONG Request

```
HTTP/1.1 404 File Not Found
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 51
content-type: text/html
expires: 0
pragma: no-cache
```

(HTTP 404 File Not Found)

Six types of response statuses

The screenshot shows a web browser window titled "Multi-Threaded Web Server (Client Side) Project Red-Crowned Crane" with the author "Yang Xikun". The interface includes input fields for "Server IP" (127.0.0.1), "Server Port" (8080), and "File Path" (/flying.pdf). Below these are two sections: "Connection Type" with radio buttons for "close" (selected) and "keep-alive (timeout = 5s)", and "Test If-Modified-Since feature (GMT time)" with a checkbox for "Enable If-Modified-Since" and a date field set to "04/20/2025, 05:22 PM". Three green buttons are visible: "Send HTTP GET Request", "Send HTTP HEAD Request", and "Send HTTP WRONG Request". The bottom section displays the response text:

```
HTTP/1.1 415 Unsupported Media Type
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 59
content-type: text/html
expires: 0
pragma: no-cache
```

(HTTP 415 Unsupported Media Type)

Handle Last-Modified and If-Modified-Since header fields

Assumptions:

- The `Last-Modified` header is returned to the Client in a 200 OK response.
- `If-Modified-Since` is an optional header of a request sent by the Client to the Server. If the Server finds this header, it will compare it with the last modified date and choose to return either 200 OK or 304 Not Modified.

Source code implementations:

- Please check the `sendOK()` method of the `HttpRequestProcessor` class. The Server's response message sent here includes the `Last-Modified` field.

```
private void sendOK(OutputStream out, String contentType, File file) throws IOException {
    String header = "HTTP/" + httpVersion + " 200 OK\r\n" +
        "Date: " + DateTimeManager.formatHttpDate(timestamp: System.currentTimeMillis()) + "\r\n" +
        "Content-Type: " + contentType + "\r\n" +
        "Content-Length: " + file.length() + "\r\n" +
        "Last-Modified: " + DateTimeManager.formatHttpDate(timestamp: file.lastModified()) + "\r\n" +
        "Cache-Control: public, max-age=0\r\n" +
        "Expires: 0\r\n" +
        "Connection: " + (keepAlive ? "keep-alive" : "close") + "\r\n" +
        "Access-Control-Allow-Origin: *\r\n" +
        "Access-Control-Allow-Methods: GET, HEAD, OPTIONS\r\n" +
        "Access-Control-Allow-Headers: Connection, If-Modified-Since\r\n" +
        "Access-Control-Expose-Headers: *\r\n" +
        "\r\n";
    out.write(b: header.getBytes());

    try (BufferedInputStream bis = new BufferedInputStream(in: new FileInputStream(file))) {
        byte[] buffer = new byte[Config.BUFFER_SIZE];
        int bytesRead;
        while ((bytesRead = bis.read(b: buffer)) != -1) {
            out.write(b: buffer, off: 0, len: bytesRead);
        }
    }

    out.flush();
}
```

Handle Last-Modified and If-Modified-Since header fields

- For the If-Modified-Since header, the server handles it in the `handleRequest()` method of the `HTTPRequestProcessor` class.

```
// Check if the file has not been modified.
long lastModified = canonicalFile.lastModified();
String lastModifiedStr = DateTimeManager.formatHttpDate( timestamp: lastModified);
String ifModifiedSinceStr = requestHeaders.getOrDefault( key: "If-Modified-Since", defaultValue: null);

if (ifModifiedSinceStr != null) {
    long ifModifiedSince = DateTimeManager.parseHttpDate( httpDate: ifModifiedSinceStr);

    // Java uses millisecond precision, but the RFC specifies second-level precision.
    long lastModifiedRFC = lastModified / 1000;
    long ifModifiedSinceRFC = ifModifiedSince / 1000;

    if (lastModifiedRFC <= ifModifiedSinceRFC) {
        sendNotModified(out, lastModifiedStr);
        return new HTTPResponse(
            serverIP: clientSocket.getLocalAddress().getHostAddress(), serverPort: clientSocket.getLocalPort(),
            clientIP: clientSocket.getInetAddress().getHostAddress(), clientPort: clientSocket.getPort(),
            datetime, method, uri, httpVersion, statusCode: 304, reasonPhrase: "Not Modified", responseBodySize: 0
        );
    }
}
```

- On the Client side, the HTTP message sent to the server will include the If-Modified-Since header. For the Clients Simulator, please refer to the `sendRequest()` method of the `ClientController` class.

```
if (ifModifiedSince != null) {
    String formattedDate = DateTimeManager.formatHttpDate( timestamp: ifModifiedSince.getTime());
    requestBuilder.append("If-Modified-Since: ").append(formattedDate).append("\r\n");
}
```

- You can also see similar logic in `client-script.js`.

```
ifModifiedSinceDateField.value = new Date().toISOString().slice(0, 16);

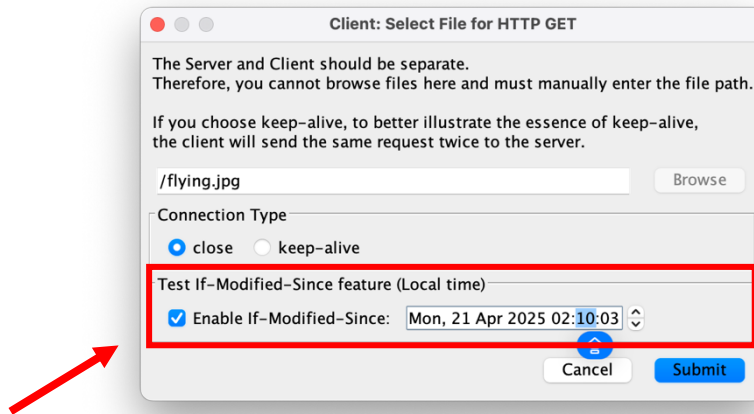
enableIMS.addEventListener( type: 'change', listener: () : void => {
    ifModifiedSinceDateField.disabled = !enableIMS.checked;
});

// The headers of the request
const headers : {Connection: any} = { 'Connection': inputs.connection };

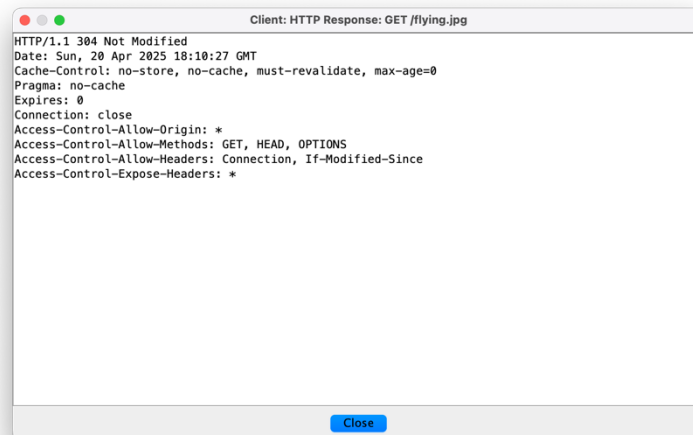
if (inputs.ifModifiedSinceDate) {
    headers['If-Modified-Since'] = inputs.ifModifiedSinceDate.toUTCString();
}
console.log(headers['If-Modified-Since'])
```

Handle Last-Modified and If-Modified-Since header fields

- When you enable "If-Modified-Since," this feature will be enabled.

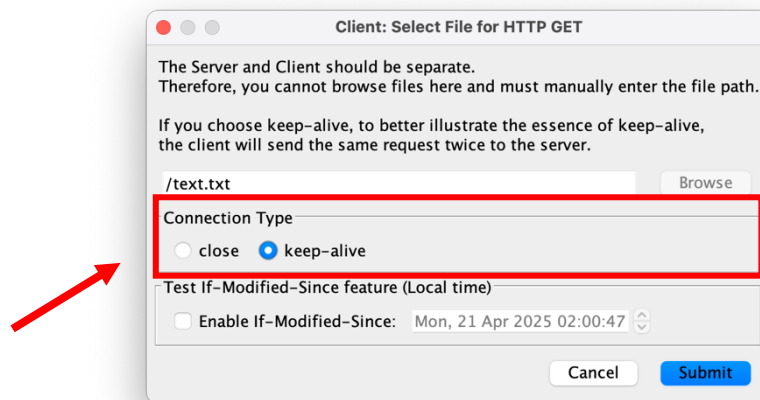


- The server will consider whether to return a 304 Not Modified response.



Handle Connection header field

- It has been pointed out that this part cannot be controlled through the browser (see **Browser Restrictions**).
- The program provide the use of the **Clients Simulator** for testing.



- If you choose a keep-alive connection, to better demonstrate the nature of keep-alive, **the program will send the same request to the server twice.**
- You can see similar records in the server log. **Using the same port confirms that two HTTP requests are using the same connection.**

```
[Welcome] Welcome to the Multi-Threaded Web Server by YANG Xikun (Project Red-Crowned Crane)
Server started on port 8080.
The thread pool is successfully started.
The server is now listening.
127.0.0.1:50584 [Sun, 20 Apr 2025 18:03:18 GMT] "GET /macaw.jpeg HTTP/1.1" 200 OK (Server: 127.0.0.1:8080) 2995130 bytes
127.0.0.1:50584 [Sun, 20 Apr 2025 18:03:19 GMT] "GET /macaw.jpeg HTTP/1.1" 200 OK (Server: 127.0.0.1:8080) 2995130 bytes
```


Handle Connection header field

- In terms of implementation, this is achieved through the cooperation between the Client and Server.
- Please specifically refer to the `HTTPResponseHandler` class, which specifies when a socket should be closed and when it should not be closed.

```
@Override
public void run() {
    try {
        // set a TIMEOUT limit for keep-alive connections
        // MAX_AWAIT_TIME is in second
        clientSocket.setSoTimeout(Config.MAX_AWAIT_TIME * 1000);

        InputStream in = clientSocket.getInputStream();
        OutputStream out = clientSocket.getOutputStream();

        boolean alive = true;
        while (alive) {
            try {
                HTTPRequestProcessor requestProcessor = new HTTPRequestProcessor(clientSocket, webRoot);
                HTTPResponse response = requestProcessor.handleRequest(in, out);

                String logMessage = response.generateLog();
                if (logMessage != null) {
                    logger.log(title: "", rawMessage: logMessage);
                }

                alive = requestProcessor.isKeepAlive();
            } catch (IOException e) {
                logger.log(title: "Warning", rawMessage: "Keep-alive connection timed out.");
                alive = false;
                logger.log(title: "Java Error: IOException", rawMessage: e.getMessage());
            }
        }

        in.close();
        out.close();
    } catch (IOException e) {
        logger.log(title: "Java Error: IOException", rawMessage: e.getMessage());
    } finally {
        try { clientSocket.close(); } catch (IOException ignored) {}
    }
}
```

Summary of Implementation Details

The programming languages / framework used in this program include:

- Java (Frontend and Backend)
- Javax Swing (Frontend)
- HTML, CSS, JS (Client Website)

The Java libraries used in this program include:

- `java.awt`
- `java.awt.event.WindowAdapter`
- `java.awt.event.WindowEvent`
- `java.awt.image.BufferedImage`
- `java.io`
- `java.io.File`
- `java.io.InputStream`
- `java.io.OutputStream`
- `java.net.InetAddress`
- `java.net.ServerSocket`
- `java.net.Socket`
- `java.net.URI`
- `java.net.URLDecoder`
- `java.nio.charset.StandardCharsets`
- `java.util`
- `java.util.ArrayList`
- `java.util.Date`
- `java.util.HashMap`
- `java.util.List`
- `java.util.Map`
- `java.util.TimeZone`
- `java.util.concurrent`
- `java.util.concurrent.TimeUnit`

Summary of Implementation Details

- javax.imageio.ImageIO
- javax.swing

The file structure of the program is:

```
/src/com/frankyang/polyu/comp2322/webserver
├── clientcontroller
│   └── ClientController.java
├── config
│   └── Config.java
├── gui
│   ├── ChooseFileDialog.java
│   ├── MainWindow.java
│   └── ResponseDialogManager.java
├── http
│   ├── HTTPRequestHandler.java
│   ├── HTTPRequestProcessor.java
│   └── HTTPResponse.java
├── logger
│   ├── Logger.java
│   └── LogListener.java
├── servercontroller
│   └── ServerController.java
├── threadpool
│   └── ServerThreadPool.java
├── util
│   └── DateTimeManager.java
└── WebServerApp.java
```

Summary of Implementation Details

You can refer to the comments at the beginning of each code file to understand its purpose. Here is also provided the general purpose of each file.

- **ClientController.java** - The Controller for the Clients Simulator, responsible for sending requests to the Server.
- **Config.java** - Defines the constants assumed within the program, for example, the default server port and the default timeout duration.
- **ChooseFileDialog.java** - Implements the GUI window for the Clients Simulator to input the file requested to the server, select the connection type, and enable or disable `If-Modified-Since`.
- **MainWindow.java** - Main window of the program. The left side is the Server Log area, and a small part on the right is the Clients Simulator.
- **ResponseDialogManager.java** - The collection of windows in the Clients side that receives responses from the Server.
- **HttpRequestHandler.java** - The multithreaded task model for HTTP requests, also controls the connection and disconnection of sockets.
- **HttpRequestProcessor.java** - Receives HTTP requests from the Clients, processes them, and then returns the corresponding response back to the Clients.
- **HttpResponse.java** - Extracts the contents of HTTP responses to facilitate logging.
- **Logger.java** - Handles web server logs.
- **LogListener.java** - A listener interface. When the Logger receives a new log, it broadcasts to all `LogListener` objects, allowing them to obtain the latest log entry.
- **ServerController.java** - The Controller of the Server, responsible for controlling the connection and disconnection of the Server socket.

Summary of Implementation Details

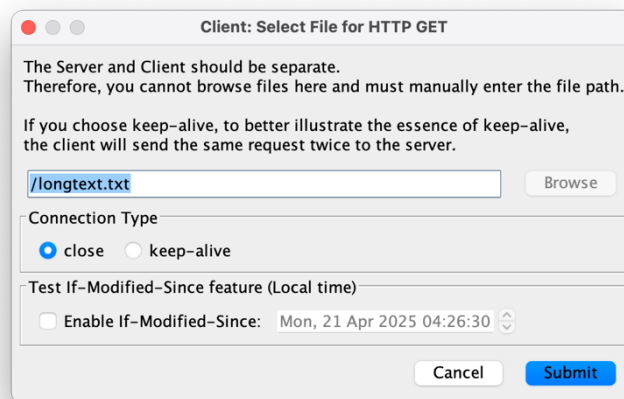
- **ServerThreadPool.java** - Defines the thread pool. The `ServerController` class holds the thread pool object. When the socket receives a new request, it will request a new thread from the thread pool to handle the HTTP request and return the response. This class uses `ExecutorService` internally.
- **DateTimeManager.java** - Defines the datetime format for this web server, also includes some methods for converting datetime formats.
- **WebServerApp.java** - Entry point of the program.

The implementation for the Client Website is a very standard HTML-CSS-JavaScript implementation. I did **not** use any external frameworks.

Complete Journey I – Testing All Functionalities

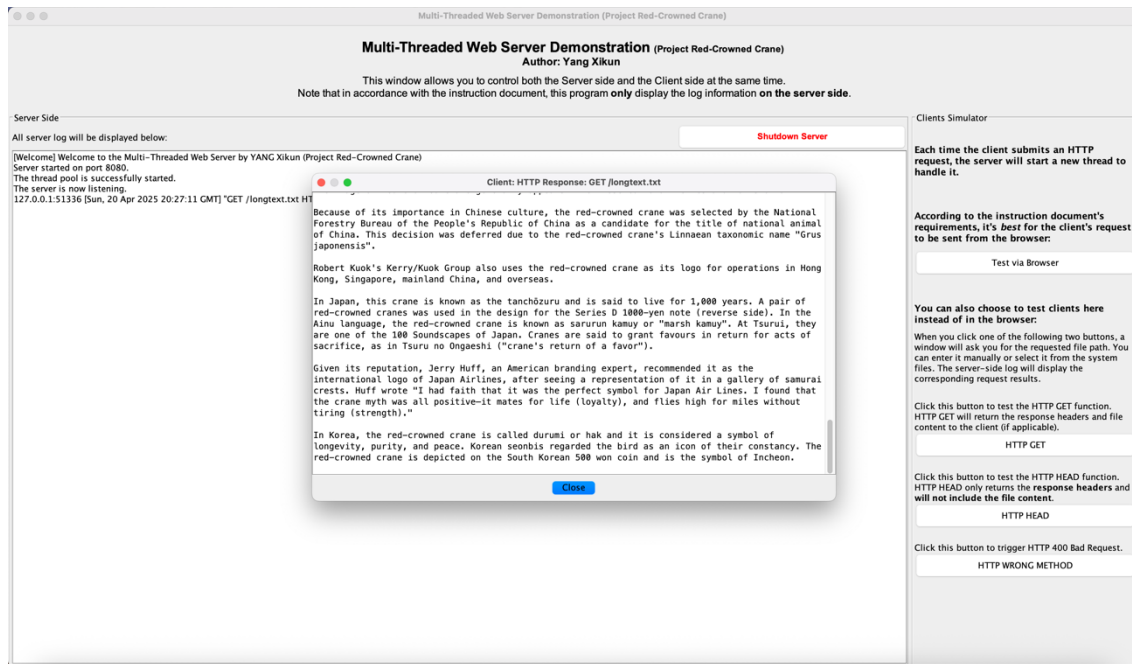


(Open the Web Server and click **HTTP GET** button)

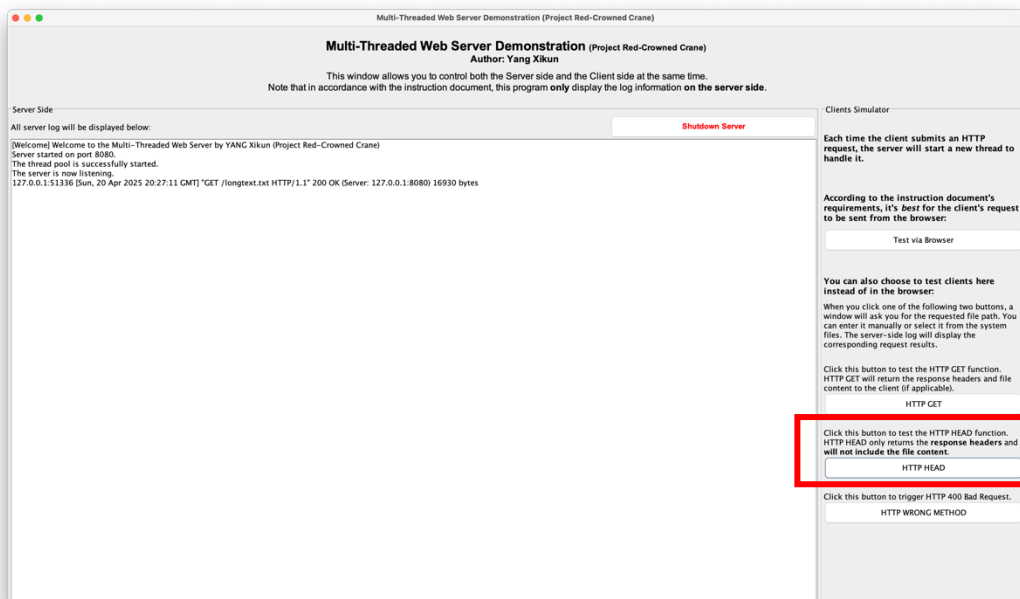


(Enter `/longtext.txt` as file path, then click **Submit**)

Complete Journey I – Testing All Functionalities

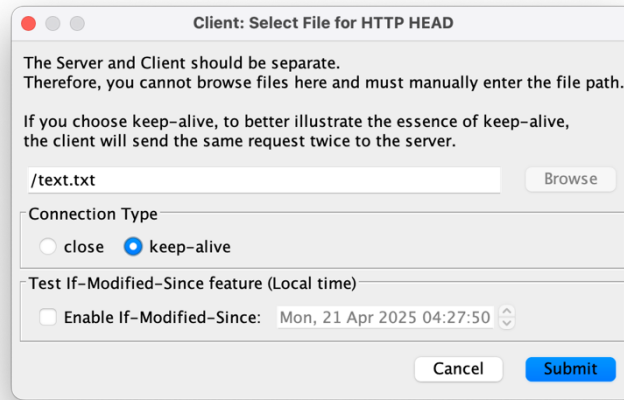


(The response and the log message. In this response, 200 OK is triggered)



(Click **HTTP HEAD** button)

Complete Journey I – Testing All Functionalities



Client: Select File for HTTP HEAD

The Server and Client should be separate.
Therefore, you cannot browse files here and must manually enter the file path.

If you choose keep-alive, to better illustrate the essence of keep-alive,
the client will send the same request twice to the server.

/text.txt

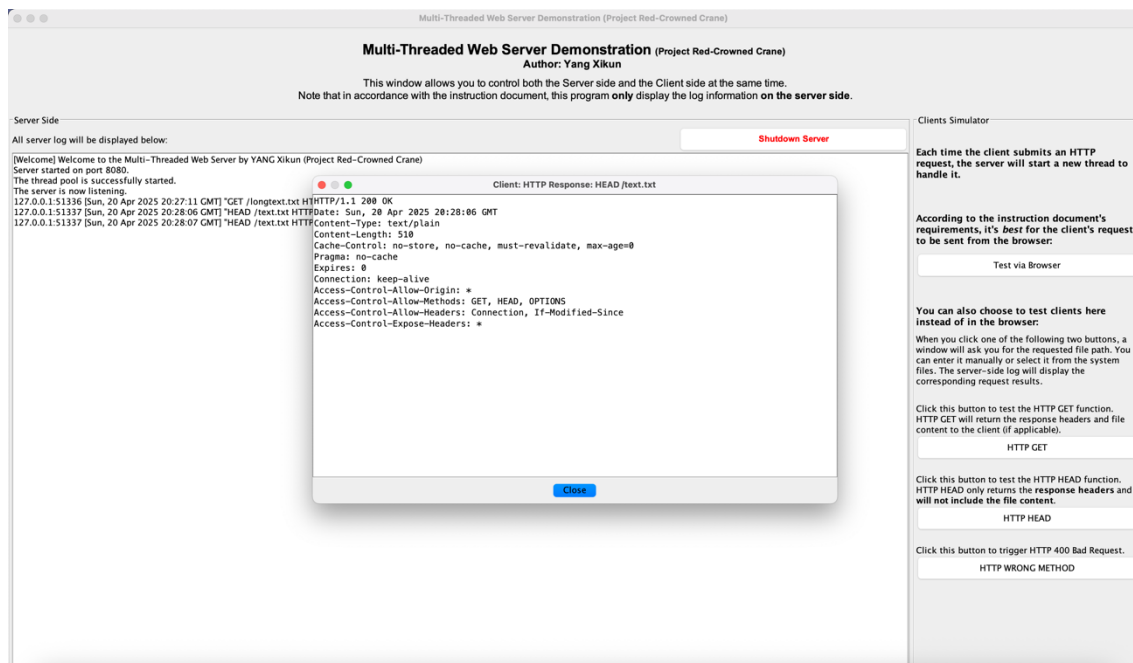
Connection Type

☐ close ☒ keep-alive

Test If-Modified-Since feature (Local time)

☐ Enable If-Modified-Since: Mon, 21 Apr 2025 04:27:50

(Enter `/text.txt` as file path and enable keep-alive connection, then click **Submit**)



Multi-Threaded Web Server Demonstration (Project Red-Crowned Crane)
Author: Yang Xikun

This window allows you to control both the Server side and the Client side at the same time.
Note that in accordance with the instruction document, this program only display the log information on the server side.

Server Side

All server log will be displayed below:

```
[Welcome] Welcome to the Multi-Threaded Web Server by YANG Xikun (Project Red-Crowned Crane)
Server started on port 8080.
The thread pool is successfully started.
The server is now listening.
127.0.0.1:51337 [Sun, 20 Apr 2025 20:27:11 GMT] "GET /longtext.txt HTTP/1.1" 200 OK
127.0.0.1:51337 [Sun, 20 Apr 2025 20:28:06 GMT] "HEAD /text.txt HTTP/1.1" 200 OK
127.0.0.1:51337 [Sun, 20 Apr 2025 20:28:07 GMT] "HEAD /text.txt HTTP/1.1" 200 OK
```

Client: HTTP Response: HEAD /text.txt

Cache-Control: no-store, no-cache, must-revalidate, max-age=0
Pragma: no-cache
Expires: 0
Connection: keep-alive
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, HEAD, OPTIONS
Access-Control-Allow-Headers: Connection, If-Modified-Since
Access-Control-Expose-Headers: *

Clients Simulator

Each time the client submits an HTTP request, the server will start a new thread to handle it.

According to the instruction document's requirements, it's best for the client's request to be sent from the browser:

You can also choose to test clients here instead of in the browser:

When you click one of the following two buttons, a window will ask you for the requested file path. You can enter it manually or select it from the system files. The server-side log will display the corresponding request results.

Click this button to test the HTTP GET function.
HTTP GET will return the response headers and file content to the client (if applicable).

Click this button to test the HTTP HEAD function.
HTTP HEAD only returns the response headers and will not include the file content.

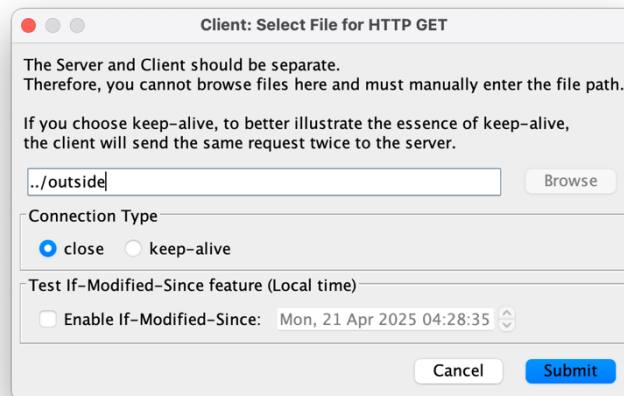
Click this button to trigger HTTP 400 Bad Request.

(Two responses and the log messages)

Complete Journey I – Testing All Functionalities

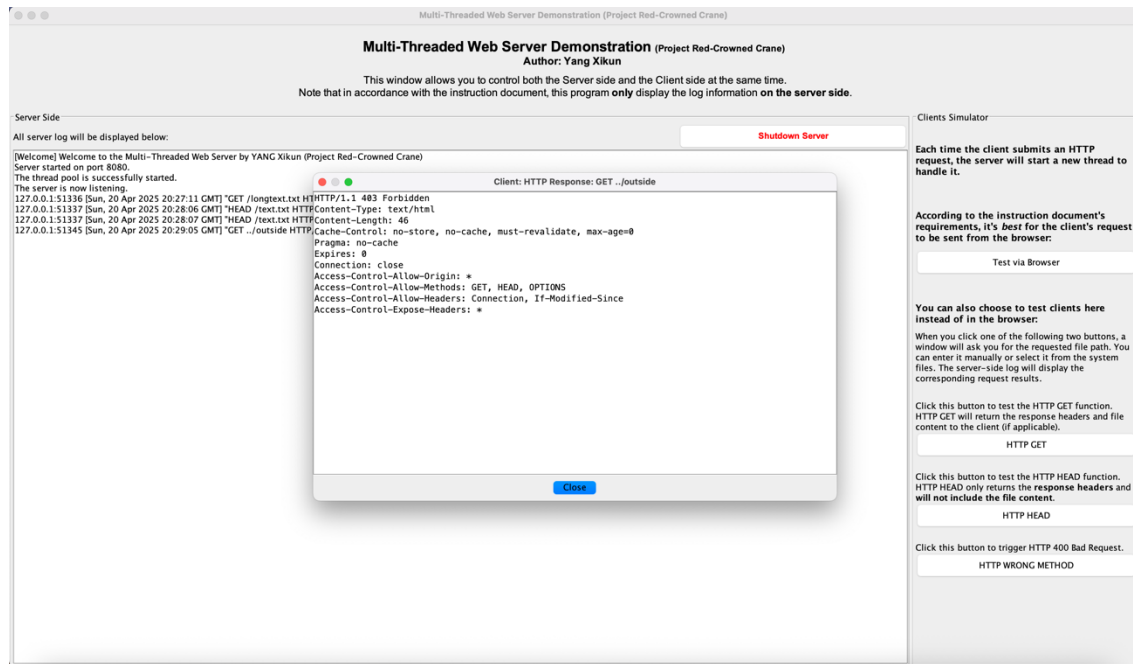


(Click HTTP GET button)

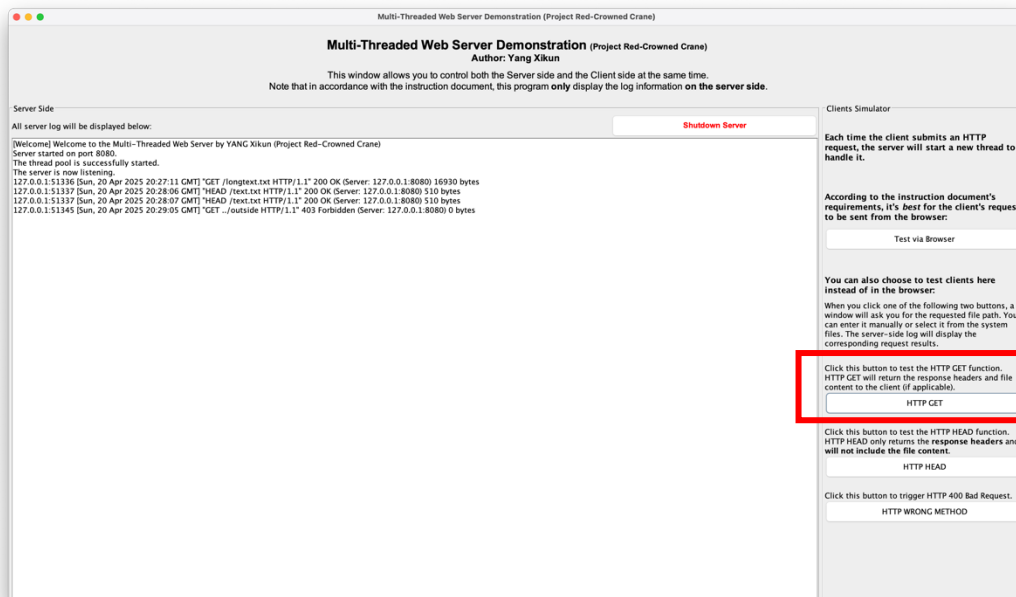


(Enter ../outside as file path, then click **Submit**)

Complete Journey I – Testing All Functionalities

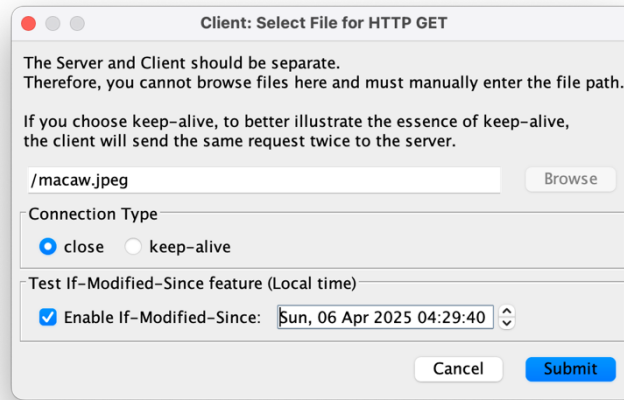


(The response and the log message. In this response, 403 Forbidden error is triggered)



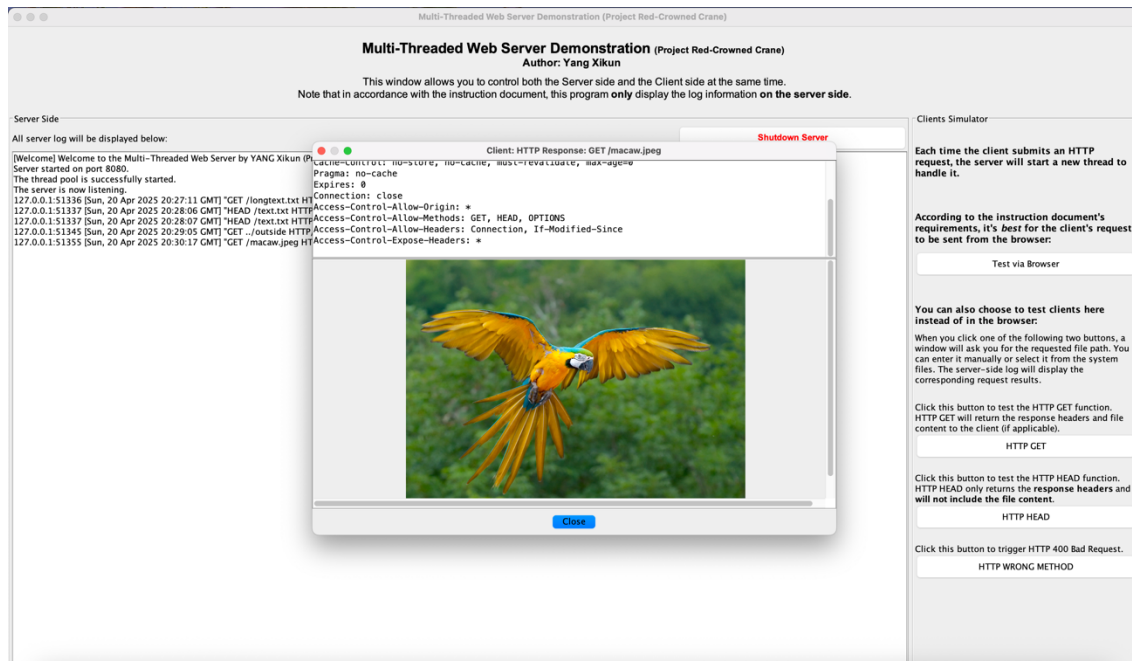
(Click HTTP GET button)

Complete Journey I – Testing All Functionalities



A dialog box titled "Client: Select File for HTTP GET". It contains instructions: "The Server and Client should be separate. Therefore, you cannot browse files here and must manually enter the file path." and "If you choose keep-alive, to better illustrate the essence of keep-alive, the client will send the same request twice to the server." There is a text input field with "/macaw.jpeg" and a "Browse" button. Below is a "Connection Type" section with radio buttons for "close" (selected) and "keep-alive". Then a "Test If-Modified-Since feature (Local time)" section with a checked checkbox "Enable If-Modified-Since:" and a date/time picker set to "Sun, 06 Apr 2025 04:29:40". At the bottom are "Cancel" and "Submit" buttons.

(Enter `/macaw.jpeg` as file path, enable If-Modified-Since and set the date to Apr 06, 2025, then click **Submit**)



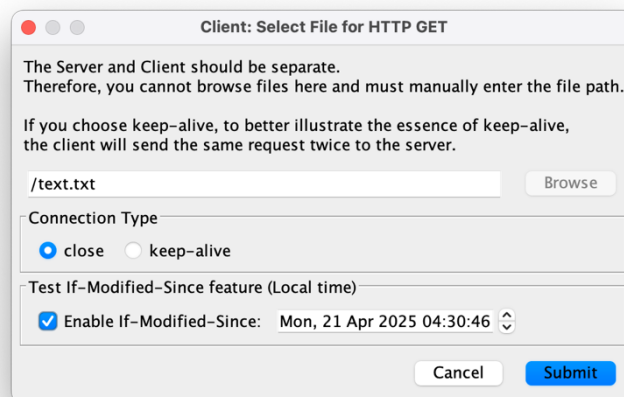
The main application window titled "Multi-Threaded Web Server Demonstration (Project Red-Crowned Crane)" by Yang Xikun. It features a "Server Side" log on the left showing timestamps and request details. A central window displays an image of a yellow and blue parrot. On the right, a "Clients Simulator" panel includes instructions and buttons for "Test via Browser", "HTTP GET", "HTTP HEAD", and "HTTP WRONG METHOD". A "Shutdown Server" button is located at the top right of the main window.

(The response and the log message, since the last modified date is later than the if-modified-since date, the image will be sent)

Complete Journey I – Testing All Functionalities

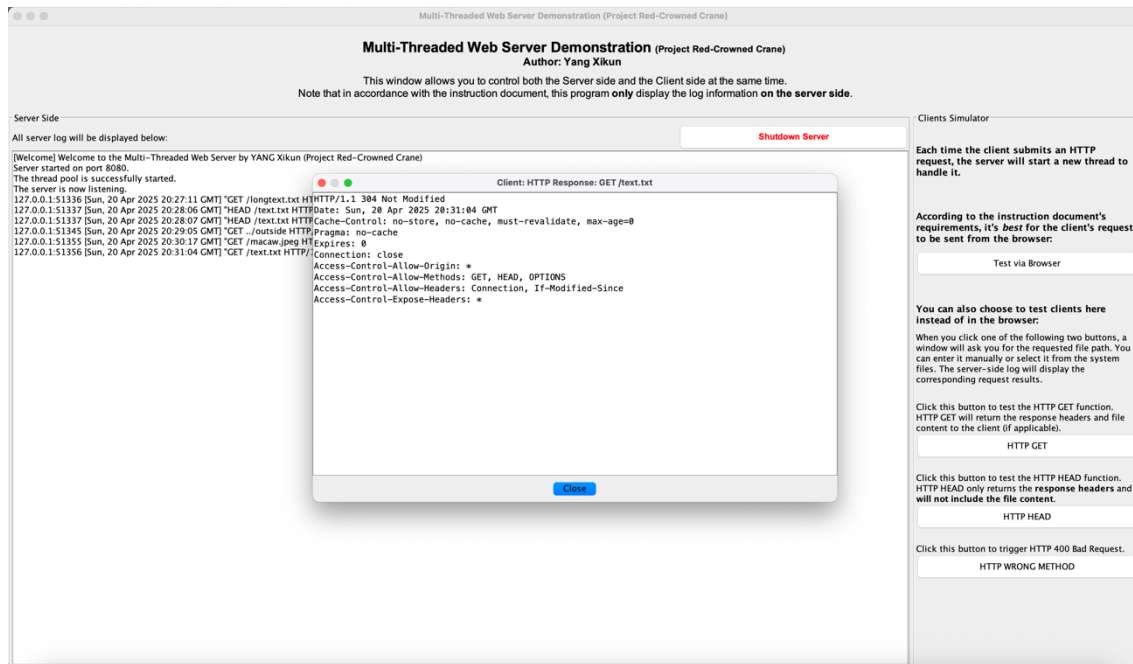


(Click **HTTP GET** button)

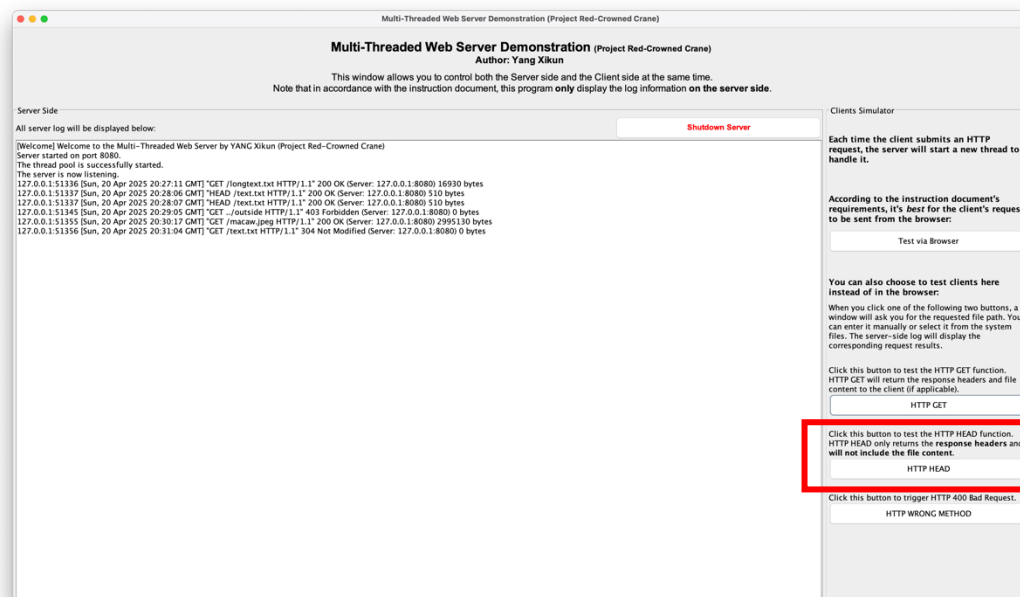


(Enter /text.txt as file path and enable If-Modified-Since, do not change the date and time, then click **Submit**)

Complete Journey I – Testing All Functionalities

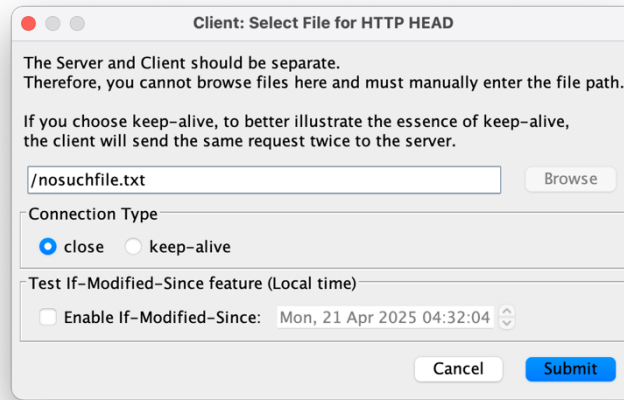


(The response and the log message. In this response, 304 Not Modified is triggered)



(Click HTTP HEAD button)

Complete Journey I – Testing All Functionalities



Client: Select File for HTTP HEAD

The Server and Client should be separate.
Therefore, you cannot browse files here and must manually enter the file path.

If you choose keep-alive, to better illustrate the essence of keep-alive,
the client will send the same request twice to the server.

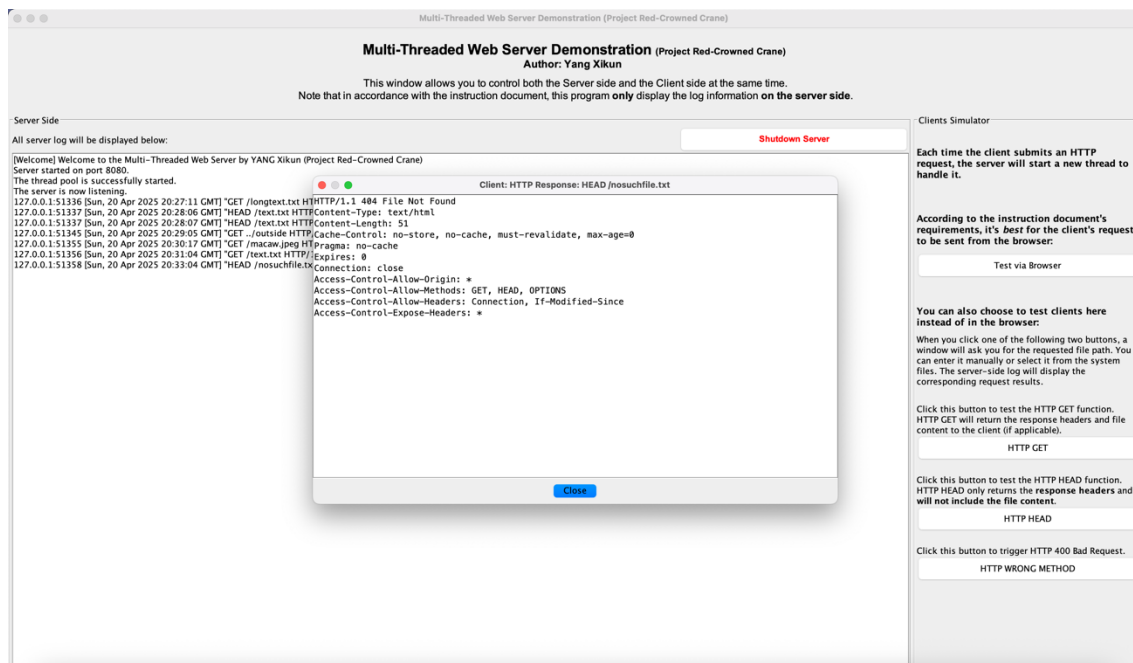
Connection Type

☒ close ☐ keep-alive

Test If-Modified-Since feature (Local time)

☐ Enable If-Modified-Since:

(Enter `/nosuchfile.txt` as file path. There is no such file under the `resources/` directory. Then click **Submit**)



Multi-Threaded Web Server Demonstration (Project Red-Crowned Crane)
Author: Yang Xikun

This window allows you to control both the Server side and the Client side at the same time.
Note that in accordance with the instruction document, this program only display the log information on the server side.

Server Side

All server log will be displayed below:

```
[Welcome] Welcome to the Multi-Threaded Web Server by YANG Xikun (Project Red-Crowned Crane)
Server started on port 8080.
The thread pool is successfully started.
The server is now listening.
127.0.0.1:51336 [Sun, 20 Apr 2025 20:27:11 GMT] "GET /longtext.txt HTTP/1.1 404 File Not Found
127.0.0.1:51337 [Sun, 20 Apr 2025 20:28:06 GMT] "HEAD /text.txt HTTP/1.1 200 OK Content-Type: text/html
127.0.0.1:51337 [Sun, 20 Apr 2025 20:28:07 GMT] "HEAD /text.txt HTTP/1.1 200 OK Content-Length: 51
127.0.0.1:51345 [Sun, 20 Apr 2025 20:29:05 GMT] "GET -/outside HTTP/1.1 200 OK Cache-Control: no-store, no-cache, must-revalidate, max-age=0
127.0.0.1:51355 [Sun, 20 Apr 2025 20:30:17 GMT] "GET /macaw.jpeg HTTP/1.1 200 OK Pragma: no-cache
127.0.0.1:51356 [Sun, 20 Apr 2025 20:31:04 GMT] "GET /text.txt HTTP/1.1 200 OK Expires: 0
127.0.0.1:51358 [Sun, 20 Apr 2025 20:33:04 GMT] "HEAD /nosuchfile.txt HTTP/1.1 404 File Not Found
Connection: close
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, HEAD, OPTIONS
Access-Control-Allow-Headers: Connection, If-Modified-Since
Access-Control-Expose-Headers: *
```

Shutdown Server

Clients Simulator

Each time the client submits an HTTP request, the server will start a new thread to handle it.

According to the instruction document's requirements, it's best for the client's request to be sent from the browser:

You can also choose to test clients here instead of in the browser:

When you click one of the following two buttons, a window will ask you for the requested file path. You can enter it manually or select it from the system files. The server-side log will display the corresponding request results.

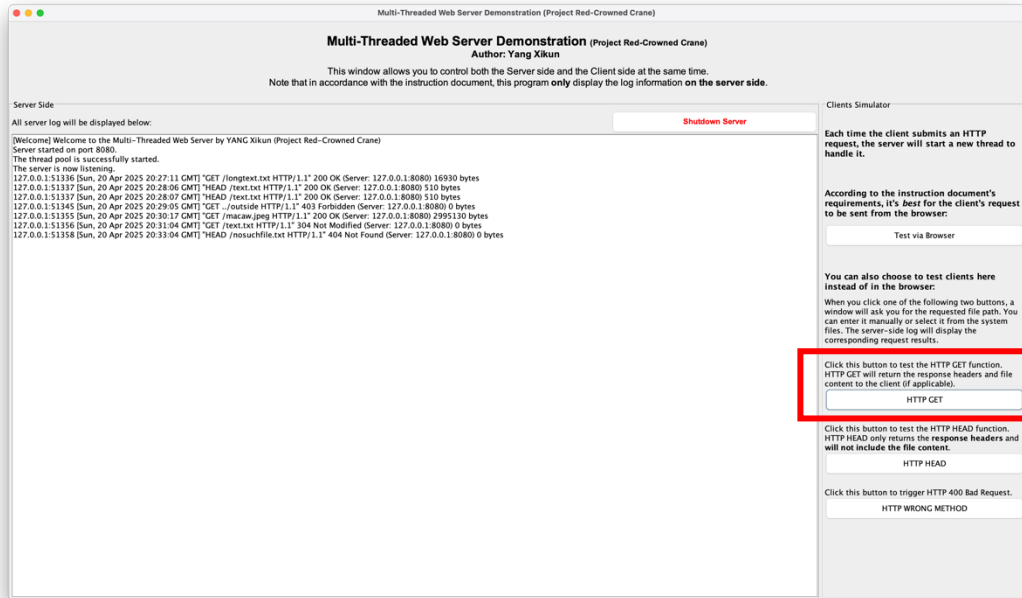
Click this button to test the HTTP GET function. HTTP GET will return the response headers and file content to the client (if applicable).

Click this button to test the HTTP HEAD function. HTTP HEAD only returns the response headers and will not include the file content.

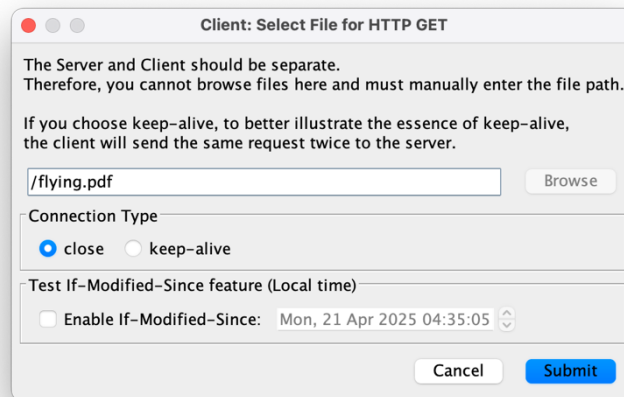
Click this button to trigger HTTP 400 Bad Request.

(The response and the log message. In this response, 404 File Not Found is triggered)

Complete Journey I – Testing All Functionalities

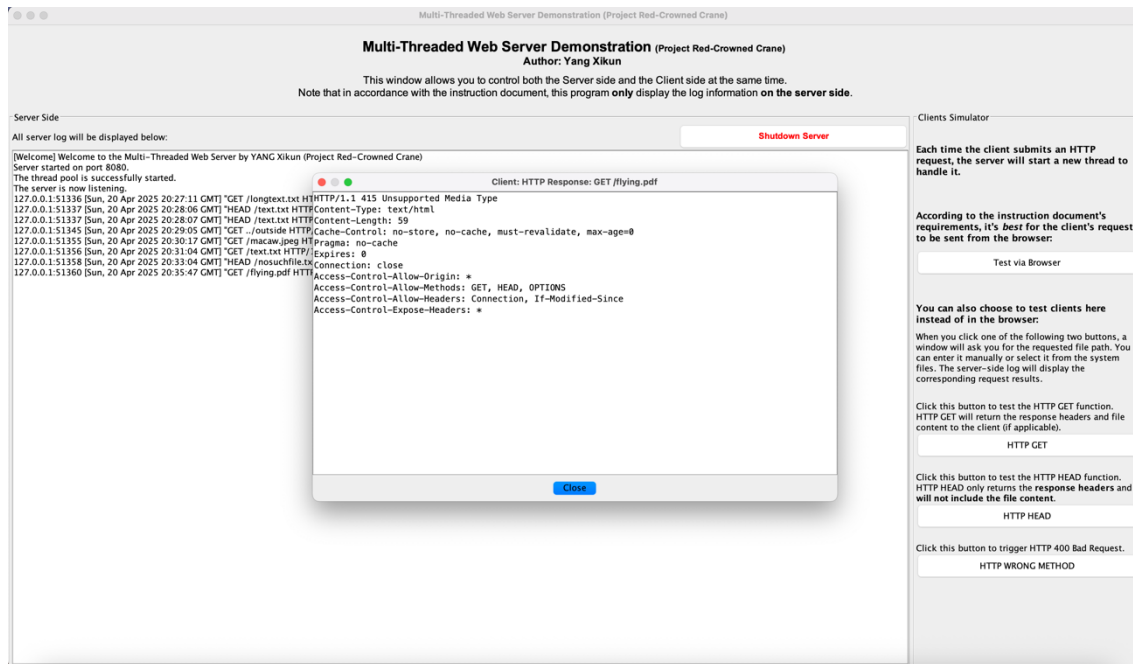


(Click HTTP GET button)

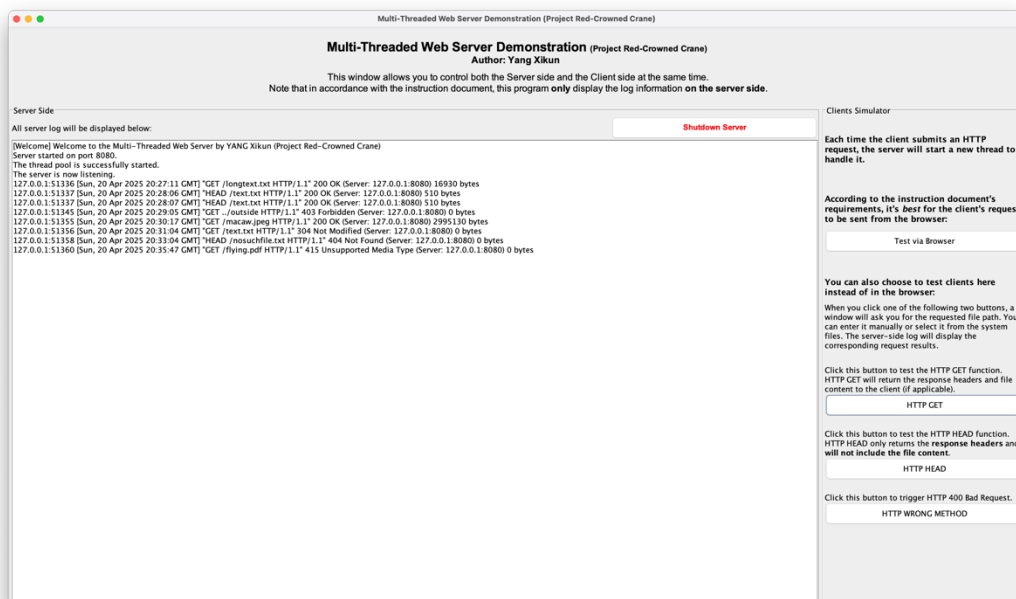


(Enter /flying.pdf as file path, then click Submit)

Complete Journey I – Testing All Functionalities

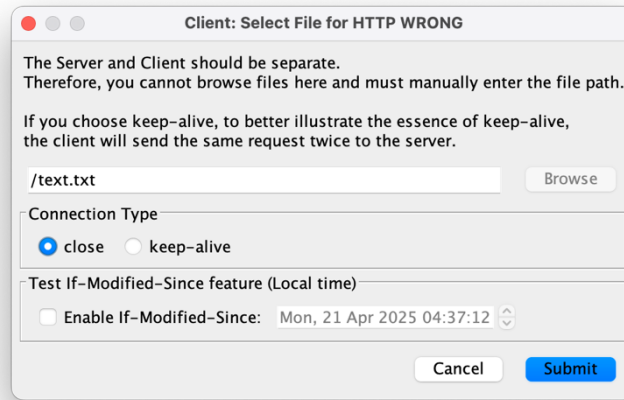


(The response and the log message. In this response, 415 Unsupported Media Type is triggered)



(Click HTTP WRONG METHOD button)

Complete Journey I – Testing All Functionalities



Client: Select File for HTTP WRONG

The Server and Client should be separate.
Therefore, you cannot browse files here and must manually enter the file path.

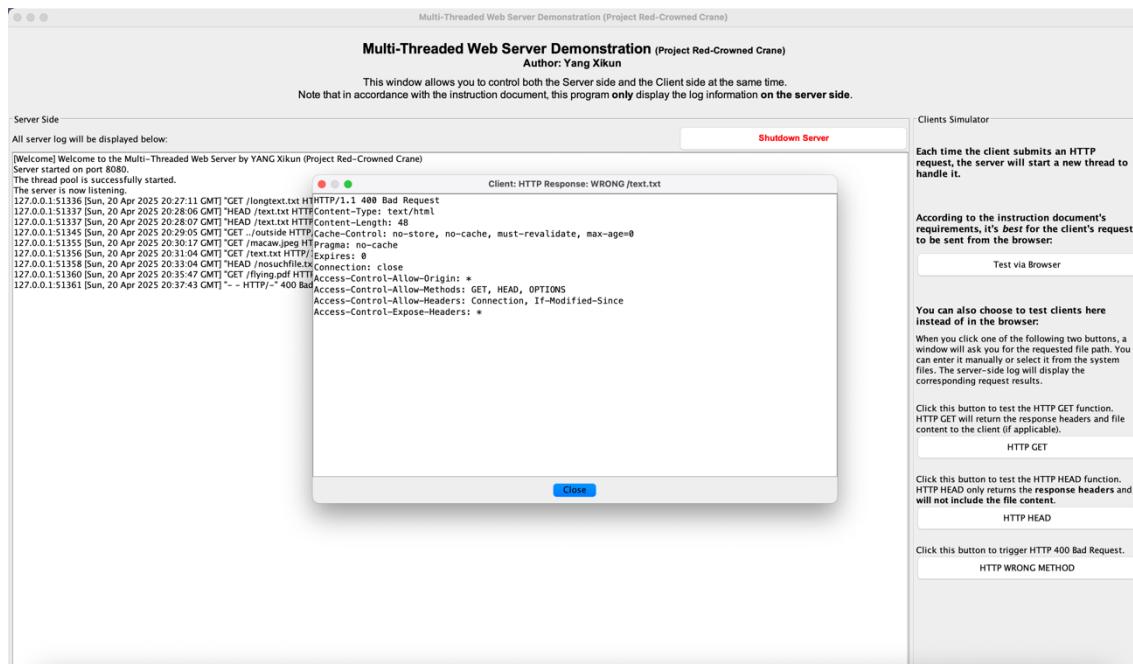
If you choose keep-alive, to better illustrate the essence of keep-alive,
the client will send the same request twice to the server.

/text.txt

Connection Type
☒ close ☐ keep-alive

Test If-Modified-Since feature (Local time)
☐ Enable If-Modified-Since: Mon, 21 Apr 2025 04:37:12

(Do not need to change any field, click **Submit**)



Multi-Threaded Web Server Demonstration (Project Red-Crowned Crane)
Author: Yang Xikun

This window allows you to control both the Server side and the Client side at the same time.
Note that in accordance with the instruction document, this program **only** display the log information on the **server side**.

Server Side

All server log will be displayed below:

```
[Welcome] Welcome to the Multi-Threaded Web Server by YANG Xikun (Project Red-Crowned Crane)
Server started on port 8080.
The thread pool is successfully started.
The server is now listening.
127.0.0.1:51336 [Sun, 20 Apr 2025 20:27:11 GMT] "GET /longtext.txt HTTP/1.1 400 Bad Request
127.0.0.1:51337 [Sun, 20 Apr 2025 20:28:06 GMT] "HEAD /text.txt HTTP/1.1 200 OK
127.0.0.1:51337 [Sun, 20 Apr 2025 20:28:07 GMT] "HEAD /text.txt HTTP/1.1 200 OK
127.0.0.1:51345 [Sun, 20 Apr 2025 20:29:05 GMT] "GET ./outside HTTP/1.1 404 Not Found
127.0.0.1:51355 [Sun, 20 Apr 2025 20:30:17 GMT] "GET /macaw.jpeg HTTP/1.1 200 OK
127.0.0.1:51356 [Sun, 20 Apr 2025 20:31:04 GMT] "GET /text.txt HTTP/1.1 200 OK
127.0.0.1:51358 [Sun, 20 Apr 2025 20:33:04 GMT] "HEAD /nosuchfile HTTP/1.1 404 Not Found
127.0.0.1:51360 [Sun, 20 Apr 2025 20:35:47 GMT] "GET /flying.pdf HTTP/1.1 200 OK
127.0.0.1:51361 [Sun, 20 Apr 2025 20:37:43 GMT] "-" HTTP/1.1 400 Bad Request
```

Client: HTTP Response: WRONG /text.txt

Shutdown Server

Clients Simulator

Each time the client submits an HTTP request, the server will start a new thread to handle it.

According to the instruction document's requirements, it's best for the client's request to be sent from the browser:

You can also choose to test clients here instead of in the browser:

When you click one of the following two buttons, a window will ask you for the requested file path. You can enter it manually or select it from the system files. The server-side log will display the corresponding request results.

Click this button to test the HTTP GET function. HTTP GET will return the response headers and file content to the client (if applicable).

Click this button to test the HTTP HEAD function. HTTP HEAD only returns the response headers and will not include the file content.

Click this button to trigger HTTP 400 Bad Request.

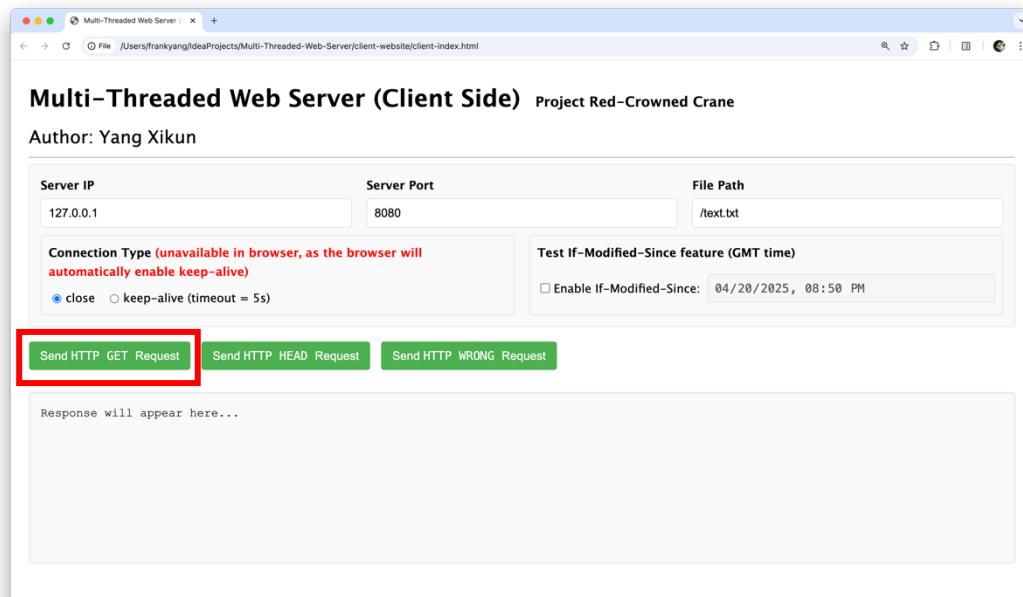
(The response and the log message. In this response, 400 Bad Request is triggered)

We have tested the GET and HEAD methods, six types of response status, Last-Modified and If-Modified-Since, as well as keep-alive and close connection.

Complete Journey II – Testing Most Functionalities



(Open the Web Server and click **Test via Browser** button)



(Client Website is already open. Click the **Send HTTP GET Request** button.)

Complete Journey II – Testing Most Functionalities

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /text.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is disabled. The 'Send HTTP GET Request' button is highlighted. The response area shows a 200 OK status with various headers including 'access-control-allow-headers', 'access-control-allow-methods', 'access-control-allow-origin', 'access-control-expose-headers', 'cache-control', 'connection', 'content-length', 'content-type', 'date', and 'expires'.

```
HTTP/1.1 200 OK
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 510
content-type: text/plain
date: Sun, 20 Apr 2025 20:52:31 GMT
expires: 0
last-modified: Sun, 20 Apr 2025 20:52:31 GMT
```

(In this response, 200 OK is triggered)

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /longtext.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is enabled with the date '04/20/2025, 08:50 PM'. The 'Send HTTP HEAD Request' button is highlighted. The response area shows a 200 OK status with various headers including 'access-control-allow-headers', 'access-control-allow-methods', 'access-control-allow-origin', 'access-control-expose-headers', 'cache-control', 'connection', 'content-length', 'content-type', 'date', 'expires', and 'last-modified'.

```
HTTP/1.1 200 OK
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 510
content-type: text/plain
date: Sun, 20 Apr 2025 20:52:31 GMT
expires: 0
last-modified: Sat, 19 Apr 2025 16:36:36 GMT
```

(Enter /longtext.txt as file path, enable If-Modified Since but do not change the date time, then click **Send HTTP HEAD Request** button)

Complete Journey II – Testing Most Functionalities

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /longtext.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is enabled with a date of 04/20/2025, 08:50 PM. The 'Send HTTP GET Request' button is highlighted. The response area shows an HTTP 1.1 304 Not Modified status with various headers including access-control-allow-headers, access-control-allow-methods, access-control-allow-origin, access-control-expose-headers, cache-control, connection, date, expires, and pragma.

```
HTTP/1.1 304 Not Modified
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
date: Sun, 20 Apr 2025 20:55:36 GMT
expires: 0
pragma: no-cache
```

(In this response, 304 Not Modified is triggered)

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /forbidden.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is disabled. The 'Send HTTP GET Request' button is highlighted. The response area shows an HTTP 1.1 304 Not Modified status with various headers including access-control-allow-headers, access-control-allow-methods, access-control-allow-origin, access-control-expose-headers, cache-control, connection, date, expires, and pragma.

```
HTTP/1.1 304 Not Modified
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
date: Sun, 20 Apr 2025 20:55:36 GMT
expires: 0
pragma: no-cache
```

(Enter /forbidden.txt as file path, disable If-Modified Since, then click **Send HTTP GET Request** button)

Complete Journey II – Testing Most Functionalities

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /forbidden.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is disabled. The 'Send HTTP GET Request' button is highlighted. The response area shows a 403 Forbidden error with the following headers:

```
HTTP/1.1 403 Forbidden
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 46
content-type: text/html
expires: 0
pragma: no-cache
```

(In this response, 403 Forbidden error is triggered)

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /flying.pdf. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is disabled. The 'Send HTTP GET Request' button is highlighted. The response area shows a 403 Forbidden error with the following headers:

```
HTTP/1.1 403 Forbidden
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 46
content-type: text/html
expires: 0
pragma: no-cache
```

(Enter /flying.pdf as file path, disable If-Modified Since, then click **Send HTTP GET Request** button)

Complete Journey II – Testing Most Functionalities

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /flying.pdf. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is disabled. The 'Send HTTP GET Request' button is highlighted. The response area shows an HTTP 415 error: 'HTTP/1.1 415 Unsupported Media Type'. The response headers are: 'access-control-allow-headers: Connection, If-Modified-Since', 'access-control-allow-methods: GET, HEAD, OPTIONS', 'access-control-allow-origin: *', 'access-control-expose-headers: *', 'cache-control: no-store, no-cache, must-revalidate, max-age=0', 'connection: keep-alive', 'content-length: 59', 'content-type: text/html', 'expires: 0', and 'pragma: no-cache'.

(In this response, 415 Unsupported Media Type error is triggered)

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /nosuchfile.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is disabled. The 'Send HTTP GET Request' button is highlighted. The response area shows an HTTP 415 error: 'HTTP/1.1 415 Unsupported Media Type'. The response headers are: 'access-control-allow-headers: Connection, If-Modified-Since', 'access-control-allow-methods: GET, HEAD, OPTIONS', 'access-control-allow-origin: *', 'access-control-expose-headers: *', 'cache-control: no-store, no-cache, must-revalidate, max-age=0', 'connection: keep-alive', 'content-length: 59', 'content-type: text/html', 'expires: 0', and 'pragma: no-cache'.

(Enter /nosuchfile.txt as file path, disable If-Modified Since, then click
Send HTTP GET Request button)

Complete Journey II – Testing Most Functionalities

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /nosuchfile.txt. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is disabled. The 'Send HTTP GET Request' button is highlighted. The response area shows an HTTP/1.1 404 File Not Found error with various headers.

```
HTTP/1.1 404 File Not Found
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 51
content-type: text/html
expires: 0
pragma: no-cache
```

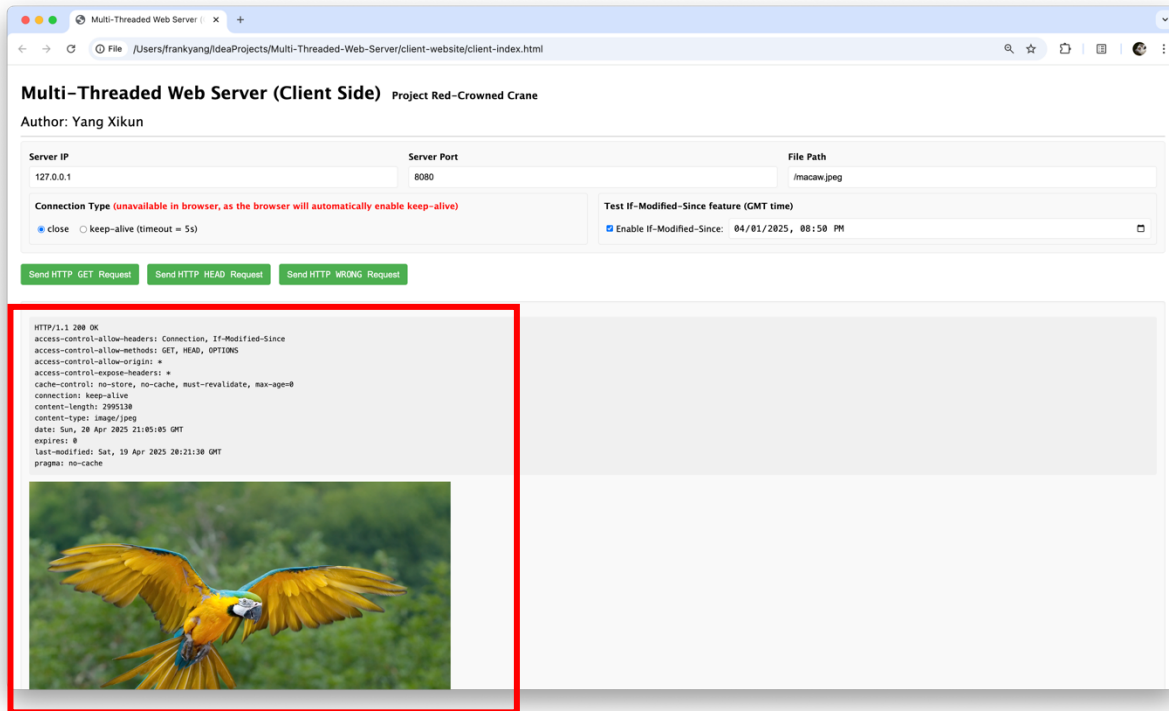
(In this response, 404 File Not Found error is triggered)

The screenshot shows the 'Multi-Threaded Web Server (Client Side)' interface. The 'Server IP' is 127.0.0.1, 'Server Port' is 8080, and 'File Path' is /macaw.jpeg. The 'Connection Type' is set to 'close'. The 'Test If-Modified-Since feature (GMT time)' is enabled with the date 04/01/2025, 08:50 PM. The 'Send HTTP GET Request' button is highlighted. The response area shows an HTTP/1.1 404 File Not Found error with various headers.

```
HTTP/1.1 404 File Not Found
access-control-allow-headers: Connection, If-Modified-Since
access-control-allow-methods: GET, HEAD, OPTIONS
access-control-allow-origin: *
access-control-expose-headers: *
cache-control: no-store, no-cache, must-revalidate, max-age=0
connection: keep-alive
content-length: 51
content-type: text/html
expires: 0
pragma: no-cache
```

(Enter /macaw.jpeg as file path, enable If-Modified Since and set the date time to an earlier date, then click **Send HTTP GET Request** button)

Complete Journey II – Testing Most Functionalities



(The response. Since the last modified date is later than the if-modified-since date, the image will be sent)

We have tested the GET and HEAD methods, five types of response status (except 400 Bad Request), Last-Modified and If-Modified-Since. Testing with the Client Website is not as complete as with the Clients Simulator. For reasons, please refer to Browser Restrictions.

Both logs of these two journeys are stored in the **log/** folder. You are also welcome to reproduce them.

Appendix I – RFC Standards

The implementation of this program references an amount of RFC documentations, including:

- Last-Modified header: [RFC 2616](#).
- If-Modified-Since header: [RFC 7232](#).
- Response statuses: [RFC 2616](#).
- Date time format of the log: [RFC 2616](#).
 - All HTTP date/time stamps must be represented in **Greenwich Mean Time (GMT)**, without exception. [\[RFC 2616, Section 3.3\]](#)
- Connection management: [RFC 7230](#).

Appendix II – Copyright Information

- `flying.jpg` and `flying.pdf` contain a photo downloaded from [here](#).
- `macaw.jpg` is a photo downloaded from [here](#).
- Please also refer to [this](#).